# Mikrocomputertechnik 1

Einheit 6: Das Steuerwerk

Prof. Dr.-Ing. Daniel Klünder

**Folie** 1 / 84

## Das Steuerwerk

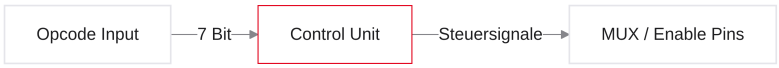
**Notizen**

Einheit 5 hat den Datenpfad bereitgestellt. Einheit 6, Mikrocomputertechnik 1, bestimmt, welcher Weg und welche Zustandsänderung zu einem Maschinenbefehl gehören. Das Steuerwerk erzeugt dafür Auswahl- und Freigabesignale. Am Ende sollen Sie die Signale für add, lw, sw und beq im Lehrsubset begründen können. RISC-V schreibt diese interne Schaltung nicht vor.
Kontrollfrage: Steuert das Steuerwerk den Assemblertext oder Bits? Bits und daraus abgeleitete Signale. Die nächste Folie zeigt einen verdrahteten, aber noch nicht passend gesteuerten Prozessor.

**Folie** 2 / 84

## Die Control Unit als Dirigent der Hardware

- Herzlich willkommen zur sechsten Vorlesung in Mikrocomputertechnik 1
- Letzte Einheit: das physikalische Rohrleitungssystem (Datenpfad)
- Multiplexer verlegt, ALU verkabelt, Speichermodule angeschlossen
- Das System ist ohne Steuersignale noch leblos
- Heute: das Gehirn der CPU — die Control Unit (das Steuerwerk)




**Notizen**

CPU bedeutet Central Processing Unit, der Prozessor. ALU bedeutet Arithmetic Logic Unit, die arithmetisch-logische Einheit. Ein MUX, Multiplexer, wählt einen Eingang. Der Datenpfad transportiert Werte. Der Registersatz hält Operanden. Der Speicher hält Daten. Das Steuerwerk entscheidet die Auswahlen.
Für add t0, t1, t2 werden zwei Register gelesen, addiert und das Ergebnis nach t0 geschrieben. Der Datenspeicher bleibt unverändert. Kein Bauteil liest den ausgeschriebenen Assemblertext. Die Dirigenten-metapher meint danach Selektor- und Freigabesignale.

Kontrollfrage: Schreibt add in den Datenspeicher? Nein. Die Agenda macht daraus Lernziele.

**Folie** 3 / 84

## Agenda für den heutigen Tag

1. Das blinde System — warum Hardware ohne Dirigent kollabiert
2. Befehlsdecodierung — wie der Chip den Opcode interpretiert
3. Die Haupt-Steuersignale — Hebel für den gesamten Datenpfad
4. ALU Control — zweistufige Decodierung (`funct3` / `funct7`)
5. Signal-Trace — die zentrale Wahrheitstabelle ausfüllen
6. Ripes-Praxis — Vorhersage-Methode („Paper & Pencil“) im Simulator




**Notizen**

Sechs Abschnitte: Datenpfad reaktivieren, Opcode lesen, Hauptsignale herleiten, ALU-Decodierung trennen, die Meistertabelle füllen und sie mit Ripes vergleichen. Opcode ist der Operationscode. ALU Control ist die nachgelagerte Operationswahl. Ein Signal-Trace verfolgt Werte und Freigaben eines Befehls. Die Frage ist nicht, ob Hardware ohne Dirigent kollabiert, sondern welche Auswahl und welche Schreibfreigabe korrekt sind. Ripes führt Befehle aus. Sein Single-Cycle-Datenpfad ist später mit diesem Lehrmodell zu vergleichen, nicht damit gleichzusetzen.
Kontrollfrage: Ist Ripes dasselbe Schaltbild wie die Folien? Nein. Zuerst reaktivieren wir den Datenpfad aus Einheit 5.

## Rückblick: Der fertige Datenpfad

- Lehrmodell nach Patterson/Hennessy, nicht dasselbe Schaltbild wie Ripes
- Register, Speicher, ALU und drei Multiplexer
- Theoretisch ausführbar: R-Type, Load, Store, Branch
- Datenkabel (meist 32 Bit) sind fest verdrahtet
- Fehlendes Puzzleteil: die Steuerleitungen „von oben“

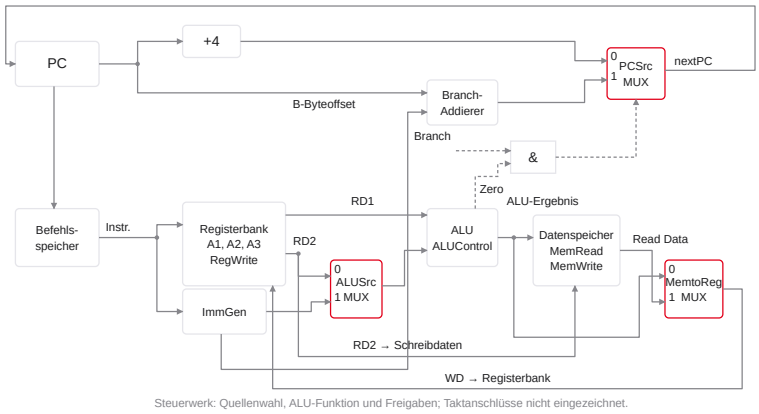



Abbildung 1: Single-Cycle-Datenpfad (vereinfacht)

### Notizen

PC bedeutet Program Counter, die Adresse der aktuellen Instruktion im Modell. Das Register File ist die Registerbank. ImmGen, Immediate Generator, erweitert den unmittelbaren Befehlswert. Die Abbildung ist das Lehrsubset: 32-Bit-Datenpfad, unkomprimierte 32-Bit-Instruktionen, C-Erweiterung aus. Unterstützt sind add, sub, and, or, lw, sw und beq. addi folgt später. bne braucht eine zusätzliche Bedingung.
Die Control Unit gibt Branch aus, nicht fertiges PCSrc. PCSrc, Program Counter Source, entsteht aus Branch und Zero. Der Store-Datenpfad führt den zweiten Registerwert zum Speicher. Der Branch-Zieladdierer ist getrennt von der ALU.
Bei add fließen zwei Registerwerte in die ALU und das Ergebnis ins Zielregister. Bei lw bildet die ALU die Adresse, der Speicher liefert den Schreibwert. Die Grafik ist nicht das Ripes-Schaltbild.
Kontrollfrage: Kommt PCSrc direkt aus der Main Control? Nein. Die festen Leitungen sagen noch nicht, welcher Pfad gilt.

**Folie** 5 / 84

## Der Datenpfad ist blind

- Kein Bauteil weiß, welchen Befehl Sie in C oder Assembler getippt haben
- Die ALU hat keine Augen für `add t0, t1, t2`
- Ein Multiplexer schaltet nicht, weil er einen `lw`-Befehl „merkt“
- Jedes Bauteil wartet auf eine physikalische Spannung (1 oder 0)
- Fehlt diese Spannung — oder ist sie falsch — kollabiert das System

```
Ein Multiplexer wählt einen Eingang:
S = 0 -> Eingang A
S = 1 -> Eingang B
Ein falscher Selektor liefert den falschen Operanden.
Das ist kein Kurzschluss und kein Don't Care.
```

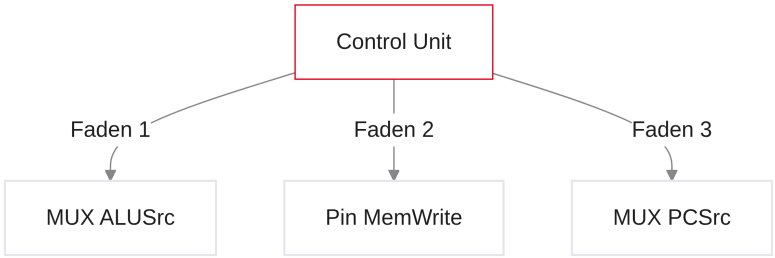
**Notizen**

Ein MUX wählt abhängig vom Selektor einen Eingang. S gleich 0 wählt A, S gleich 1 wählt B. Ein falscher Selektor liefert den falschen Operanden. Das ist keine Kollision der Daten und kein Einfrieren des Prozessors. Ein physikalisch undefinierter Eingang und ein Don’t Care der späteren Tabelle sind verschieden. Logisch 0 und 1 sind die Signalwerte; konkrete Spannungen hängen vom Bauteil ab und werden hier nicht erfunden.
Kontrollfrage: Kollidieren beide MUX-Eingänge auf der Ausgangsleitung? Nein, einer wird gewählt. Als Nächstes wird die zentrale Steuerung dieser Auswahlen veranschaulicht.

**Folie** 6 / 84

## Die Marionetten-Metapher

- Datenpfad als Marionette
- Gelenke: Multiplexer und Write-Enable-Pins
- Arme und Beine bewegen sich nicht von allein
- Puppenspieler: die Control Unit
- Kupferfäden: Steuerleitungen — im Takt gezogen, tanzt die Puppe



**Notizen**

Die Marionette ist nur der Einstieg. Ein Gelenk entspricht einem MUX, der Faden einem Selektor, die Schreibfreigabe einem Enable. Für add gilt ALUSrc gleich 0, also der zweite Registerwert, RegWrite gleich 1 und MemWrite gleich 0. ALUSrc heißt ALU Source, RegWrite Register Write, MemWrite Memory Write. Kombinatorische Selektoren müssen nicht auf die Taktflanke warten. Zustandselemente übernehmen nach ihrem Taktmodell. Im Kurs ist das die aktive Flanke.
Kontrollfrage: Wechselt ein MUX-Selektor erst bei der Flanke? Nicht zwingend. Die nächste Folie definiert das Steuerwerk als Schaltung.

**Folie** 7 / 84

## Was ist die Control Unit?

- Separate, fest verdrahtete Box aus Logikgattern auf dem Chip
- In Architekturdiagrammen oft als Wolke oder abgerundete Box
- Input: Teile des aktuellen 32-Bit-Befehlsworts
- Verarbeitung: reine Kombinatorik (AND/OR-Netzwerke)
- Output: 1-Bit- bzw. 2-Bit-Steuersignale an den Datenpfad

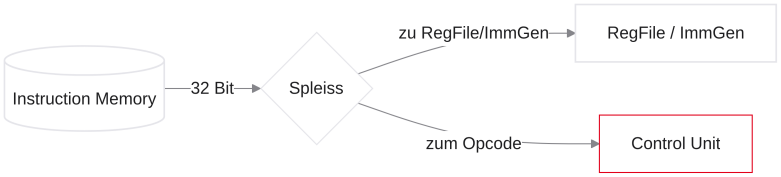
**Notizen**

Die Aussage reine Kombinatorik ohne Flip-Flops gilt für dieses Single-Cycle-Steuerwerk. Single Cycle bedeutet: ein unterstützter Befehl wird in einem Taktzyklus ausgeführt. Andere Steuerwerke können Zustand, mehrere Takte oder ein Mikroprogramm haben. Eine Box im Bild muss keine räumlich getrennte Hardware sein. Input sind Befehlsfelder, Output die benannten Steuersignale. Hardwired Logic heißt fest verdrahtete kombinatorische Logik, nicht eine RISC-V-Pflicht.
Kontrollfrage: Muss jedes Steuerwerk ohne Zustand auskommen? Nein. Als Nächstes kommt die Herkunft der Eingangsbits.

**Folie** 8 / 84

## Input: Woher kommen die Befehle?

- Der Program Counter adressiert den Befehlsspeicher (Instruction Memory)
- Der Speicher liefert 32 Bit (Phase: Instruction Fetch)
- Kabelspleiß: ein Teil wandert in den Datenpfad (Register, ImmGen)
- Ein anderer Teil wird abgezweigt in die Control Unit
- Der Puppenspieler liest physikalisch mit, welcher Code aus dem ROM kommt



**Notizen**

Der PC adressiert den Befehlsspeicher. Instruction Fetch ist das Holen des 32-Bit-Befehlsworts. Daraus werden Felder an Registerbank, ImmGen und Steuerwerk geführt. Der Befehlsspeicher muss kein ROM sein. ROM, Read-Only Memory, ist nur eine mögliche Speicherart. Ohne C-Erweiterung ist jede Basisinstruktion vier Byte. Das ganze Wort liegt an; verschiedene Blöcke lesen verschiedene Bitpositionen. Kontrollfrage: Liest das Steuerwerk das gesamte Wort als eine Zahl ohne Felder? Nein, es nutzt festgelegte Positionen. Danach betrachten wir die Ausgänge.

**Folie** 9 / 84

## Output: Die blauen Steuerkabel

- Auf der Ausgangsseite treten die Steuerleitungen aus
- Für unseren RV32I-Single-Cycle-Prozessor: sieben primäre Steuersignale
- Die meisten Signale sind 1 Bit: logischer Pegel 0 oder 1

| Kategorie | Signale |
|---|---|
| Multiplexer (Weichen) | `ALUSrc`, `MemtoReg`, Branch (für `PCSrc`) |
| Erlaubnis (Write-Enable) | `RegWrite`, `MemWrite` |
| Leseschutz | `MemRead` |
| Sondertyp (2 Bit) | `ALUOp` (grobe ALU-Steuerung) |

**Notizen**

Sieben benannte Hauptsignale ergeben acht Steuerbits, weil ALUOp zwei Bit hat. Das gilt für unseren Lehrsubset-Datenpfad, nicht für jeden RV32I-Prozessor. ALUSrc wählt den zweiten ALU-Operanden. MemtoReg, Memory to Register, wählt den Register-Schreibwert. RegWrite gibt das Registerschreiben frei. MemRead, Memory Read, gibt im Modell den Lesezugriff frei. MemWrite gibt den Schreibzugriff frei. Branch kennzeichnet einen unterstützten bedingten Sprung, ist aber nicht die fertige Sprungentscheidung. ALUOp ist die zweibitige Kategorie, nicht das ALUControl-Wort. PCSrc ist abgeleitet, kein achter Main-Control-Ausgang.
Kontrollfrage: Ist Branch der MUX-Selektor des PC? Nein, PCSrc ist Branch und Zero. Wie entstehen diese Werte zeitlich?

**Folie** 10 / 84

### Kombinatorische Logik (Das Timing)

- Wichtige Eigenschaft: Die Control Unit hat keinen eigenen Takt
- Reine kombinatorische Schaltung (wie ein großer Addierer)
- Kein Zustand, keine Flip-Flops, kein Speicher
- Nach der Gatterlaufzeit müssen die Signale vor der Übernahmeflanke stabil sein
- Der Datenpfad nutzt sie im Taktzyklus

```
/* Control Unit: rein funktional, kein State */
Steuersignale = generate_control(Instruction_Bits);
```

**Notizen**

Die Uhrlosigkeit gilt für die kombinatorische Decoderfunktion, nicht für den ganzen Prozessor. Nach einer endlichen Laufzeit müssen die Steuer- und Datenwerte vor der übernehmenden Flanke stabil sein. Kurze Übergangszustände sind möglich. Der längste relevante Pfad begrenzt die zulässige Zykluszeit. Nanosekunden ohne Messung bleiben weg. Ein Pseudocode beschreibt die Funktion und ist kein Programm, das die CPU ausführt.
Kontrollfrage: Stehen die Signale zeitlos fest? Nein, nach der Laufzeit. Eine Wahrheitstabelle spezifiziert die stabile Zuordnung.

**Folie** 11 / 84

### Die Wahrheitstabelle (Refresher)

- Kombinatorik lässt sich vollständig als Wahrheitstabelle beschreiben
- Input-Spalten: Bit-Kombinationen der Eingabe
- Output-Spalten: resultierende Steuersignale
- Die Wahrheitstabelle ist der Bauplan für den Hardware-Entwurf
- Heutige Mission: diese Matrix für unseren Prozessor ausfüllen

| Input A | Input B | Output Y (AND) | Output Z (XOR) |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| … | … | … | … |

**Notizen**

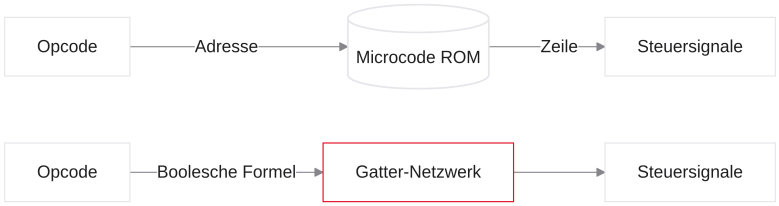
AND ist Und, OR ist Oder, XOR ist exklusives Oder. XOR ist eins bei genau einem gesetzten Eingang, OR auch bei zwei Einsen. Die vier Fälle: 00 ergibt AND 0 und XOR 0; 01 und 10 ergeben AND 0 und XOR 1; 11 ergibt AND 1 und XOR 0. Dieselbe Tabellenidee ordnet später einen Opcode einem Steuerwort zu. Die Tabelle spezifiziert die Funktion. Sie behauptet nicht, dass ebenso viele Zeilen physisch gespeichert

werden.
Kontrollfrage: Ist XOR bei zwei Einsen eins? Nein. Eine solche Funktion kann verschieden implementiert werden.

**Folie** 12 / 84

## Architektur-Design: LUT vs. Gatter

- Methode 1: Hardwired Logic — Karnaugh-Veitch, minimierte AND/OR-Gatter; schnell, aber starr
- Methode 2: Microcode (ROM / LUT) — Wahrheitstabelle als Look-up; Opcode als Adresse
- RISC-V ist prädestiniert für Hardwired Logic: kleine, effiziente Chips




**Notizen**

Drei Konzepte: kombinatorische Gatterlogik; eine Look-up Table, kurz LUT, oder eine ROM-Abbildung; ein mikroprogrammiertes Steuerwerk mit Steuerspeicher und Ablauf für interne Schritte. Eine opcode-adressierte Tabelle allein ist noch kein Mikroprogramm. FPGA bedeutet Field-Programmable Gate Array, eine konfigurierbare Logik. Ein ASIC, Application-Specific Integrated Circuit, ist eine feste Fertigung. RISC-V schreibt keine dieser Umsetzungen vor.
Kontrollfrage: Ist jede Tabelle ein Mikroprogramm? Nein. Die nächste Folie ordnet die regelmäßige Codierung als Entwurfshilfe ein.

**Folie** 13 / 84

## Der RISC-V-Vorteil

- x86-Implementierungen nutzen oft Mikrocode oder µOp-Zerlegung
- Das ist eine typische Entwurfsentscheidung, kein Beweis, dass Hardwired unmöglich ist
- Eine vollständig aufgelistete Wahrheitstabelle ist nicht die synthetisierte Fläche
- Konkrete Gatterzahl und Timing brauchen eine Implementierung und Messung

**Notizen**

Unser Modell trennt Main Control und ALU Control. Die ISA, Instruction Set Architecture, schreibt diese Zerlegung nicht vor. Regelmäßige Felder erleichtern eine strukturierte Decodierung. Sie garantieren keine Chipgröße und keine Taktfrequenz. Eine vollständig aufgelistete Tabelle ist nicht die synthetisierte Fläche. x86 kann intern Micro-Operations, kurz µOps, verwenden; das macht die sichtbare ISA nicht zu diesem Lehrdecoder.

Kontrollfrage: Folgt die Zweiteilung aus der RISC-V-Spezifikation? Nein. Der nächste Abschnitt sucht die relevanten Felder im Befehlswort.

**Folie** 14 / 84

## Befehlsdecodierung und Opcode

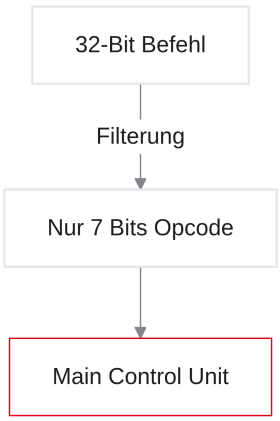
**Notizen**

Eingänge und Ausgänge sind bekannt. Jetzt wird die Eingangsinformation gezielt reduziert. Der Opcode ist der Operationscode zur Familienerkennung. Die Leitfrage lautet: Welche Bits steuern die Pfadauswahl, und welche führen Operanden oder Registeradressen? Nicht jedes Bit des Befehlsworts ist ein Steuersignal. Kontrollfrage: Decodiert der Opcode allein add gegen sub? Nein. Die nächste Folie motiviert eine strukturierte statt erschöpfende Analyse.

**Folie** 15 / 84

### Wir müssen den Befehl lesen — aber wie?

- Die Control Unit muss den Befehl interpretieren
- Müssen wirklich alle 32 Bits gleichzeitig analysiert werden?
- Antwort liefert die Anatomie der RISC-V-Formate
- Der 7-Bit-`opcode` als Identifikationsschild
- Das Konzept Don't Care ($X$) und warum es Hardware spart




**Notizen**

2 hoch 32 ist 4 294 967 296 mögliche Bitmuster. Das ist die Größe einer naiv vollständig aufgelisteten Tabelle, nicht die Gatterzahl. Eine strukturierte Funktion wertet Felder getrennt und gemeinsame Bedingungen zusammen aus. Ein X in einer späteren Tabelle erlaubt nicht, gültigkeitsrelevante Bits zu ignorieren.

Kontrollfrage: Muss ein Decoder 4 Milliarden Zeilen speichern? Nein. Die Formate zeigen, welche Felder an welcher Position stehen.

**Folie** 16 / 84

### Die Anatomie des Befehls (Refresher)

- Instruction Formats R, I, S, B aus Einheit 2
- Felder wie `rs1` (Bits 19:15) oder `rd` (Bits 11:7) sind Registeradressen
- Die Control Unit interessiert sich nicht für diese Hausnummern
- Ob `t0` oder `s11` addiert wird: MUX-Stellungen bleiben gleich
- Diese Bits können wir für das Steuerwerk ignorieren

```
31          24    19    14      11      7        0
+-------+-----+-----+-------+-----+-------------+
| funct7| rs2 | rs1 | funct3|  rd |   opcode    |
+-------+-----+-----+-------+-----+-------------+
  (Detail)  (ignoriert)  (Detail)   WICHTIG!
```

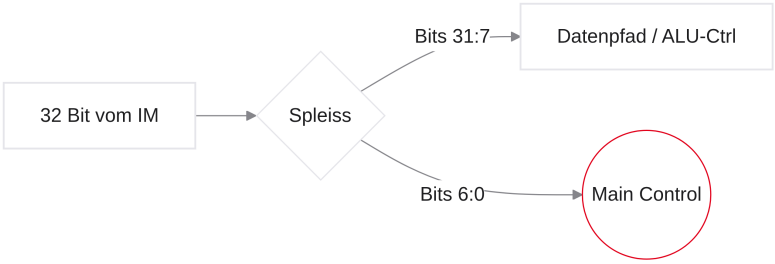
**Notizen**

R-Type: funct7 in 31 bis 25, rs2 in 24 bis 20, rs1 in 19 bis 15, funct3 in 14 bis 12, rd in 11 bis 7, opcode in 6 bis 0. rs bedeutet source register, rd destination register, funct Funktionsfeld. Die 15 Registerbits gelten nur für diesen R-Type. I-Type ersetzt funct7 und rs2 durch ein 12-Bit-Immediate. S- und B-Type haben kein rd. Ihre Bits 11 bis 7 tragen Immediate-Anteile. Der Immediate ist gesplittet und nicht ein zusammenhängendes Feld.
Kontrollfrage: Enthält sw ein rd-Feld? Nein. In allen betrachteten Formaten liegt der Opcode an derselben Position.

**Folie** 17 / 84

## Das 7-Bit-Identifikationsschild

- In jedem Format an derselben Stelle: der `opcode`
- Bits $[6:0]$ — exakt 7 Bit breit
- Diese 7 Drähte sind der Input der Main Control Unit
- Der Opcode sagt die Befehlsfamilie, nicht `add` vs. `sub`
- Detailentscheidungen folgen später über `funct3` / `funct7`




**Notizen**

Die Bits 6 bis 0 sind sieben Positionen einschließlich beider Grenzen. 2 hoch 7 ist 128 mögliche Opcode-Werte. Nicht alle werden in diesem Subset genutzt. add und sub teilen 0110011. Die konkrete Operation steht in funct3 und funct7. Nur nach dem Opcode zu entscheiden ist eine Modellwahl für die gezeigten Pfadauswahlen. Weitere Details brauchen zusätzliche Decoder.
Kontrollfrage: Reicht der Opcode, um add von sub zu unterscheiden? Nein. Zuerst folgt die Familie der Register-Rechenbefehle.

**Folie** 18 / 84

## Opcode: R-Type (Rechnen)

- Familie: zwei Register lesen, rechnen, in ein Register schreiben (`add`, `sub`, `and`, `or`, `sll`, ...)
- Opcode (binär): `0110011`
- Keine Konstanten nötig; RAM unbeteiligt; `RegWrite` zwingend

| Bit 6 | Bit 5 | Bit 4 | Bit 3 | Bit 2 | Bit 1 | Bit 0 |
|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 | 1 | 1 |

**Notizen**

0110011 ist hexadezimal 0x33. In dieser Zeile unterstützen wir add, sub, and und or mit legalen Encodings. sll ist ein echter weiterer R-Type-Befehl, aber nicht durch die aktuelle ALU-Control-Tabelle abgedeckt. t0, t1 und t2 sind Aliase temporärer Register. Lehrwerte: t1 gleich 7, t2 gleich 3, add t0, t1, t2 schreibt 10. Die Quellen bleiben unverändert.

Kontrollfrage: Deckt die Zeile jeden R-Type-Befehl ab? Nein. Load hat einen anderen Datenweg.

**Folie** 19 / 84

### Opcode: Load-Familie im I-Format

- Familie: Daten aus dem Arbeitsspeicher in die CPU (`lw`, `lb`, `lh`, ...)
- Opcode (binär): `0000011`
- Immediate für Adressberechnung; RAM lesen; Write-Back aus dem RAM

| Bit 6 | Bit 5 | Bit 4 | Bit 3 | Bit 2 | Bit 1 | Bit 0 |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 1 |

**Notizen**

0000011, hexadezimal 0x03, kennzeichnet Loads, nicht alle I-Type-Befehle. addi hat 0010011, also 0x13. lw bedeutet Load Word und lädt 32 Bit. lb und lh brauchen Breiten- und Erweiterungssteuerung, die nicht in der Grundmatrix steht. Die ALU bildet Basis plus vorzeichenerweiterten Offset. Die Adresse ist nicht der geladene Wert.
Kontrollfrage: Hat addi denselben Opcode wie lw? Nein. Store verwendet eine ähnliche Adresse und schreibt in die andere Richtung.

**Folie** 20 / 84

### Opcode: S-Type (Store)

- Familie: CPU-Daten im RAM ablegen (`sw`, `sb`, `sh`)
- Opcode (binär): `0100011`
- Immediate für Adresse; `MemWrite` aktiv; kein `rd` → `RegWrite = 0`

| Bit 6 | Bit 5 | Bit 4 | Bit 3 | Bit 2 | Bit 1 | Bit 0 |
|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 0 | 1 | 1 |

**Notizen**

sw bedeutet Store Word. Der Opcode ist 0100011, hexadezimal 0x23. Die Basis kommt aus rs1, die Store-Daten aus rs2, der Offset ist ein gesplitteter S-Immediate. Die ALU wählt den Immediate als zweiten Operanden. Der zweite Registerwert bleibt trotzdem am Speicher-Schreibdateneingang verfügbar. sb und sh brauchen zusätzliche Schreibmasken.
Kontrollfrage: Verschwindet rs2, weil ALUSrc den Immediate wählt? Nein, es liefert die Schreibdaten. Branch verändert vor allem die nächste Befehlsadresse.

**Folie** 21 / 84

## Opcode: B-Type (Branch)

- Familie: bedingte Sprünge (`beq`, `bne`, `blt`, ...)
- Opcode (binär): `1100011`
- Registervergleich; nichts schreiben; Sprungentscheidung mit ALU-Flag

| Bit 6 | Bit 5 | Bit 4 | Bit 3 | Bit 2 | Bit 1 | Bit 0 |
|---|---|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 0 | 1 | 1 |

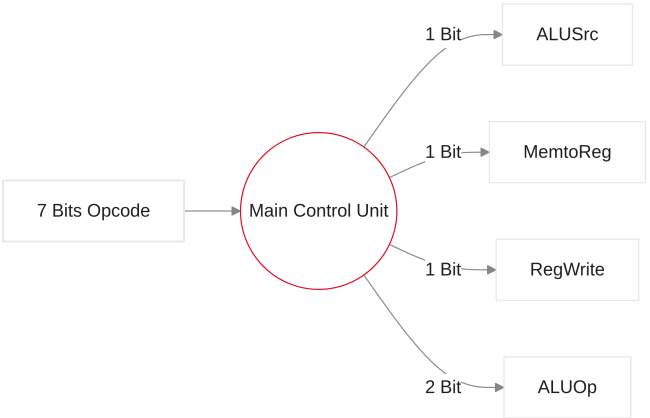
**Notizen**

beq bedeutet Branch if Equal, bne Branch if Not Equal, blt Branch if Less Than. Der Opcode 1100011, hexadezimal 0x63, erkennt die Familie. funct3 unterscheidet die Bedingung. Das bisherige Zero-Netz gilt für beq. bne braucht die negierte Gleichheit. blt und bltu unterscheiden signed und unsigned. Zero ist kein architektonisches Flag. RegWrite und MemWrite bleiben im Modell null. Der PC wird trotzdem aktualisiert. Kontrollfrage: Deckt Branch und Zero auch bne ab? Nein. Die Familienerkennung wird in der Main-Control-Box zusammengefasst.

**Folie** 22 / 84

## Der Main-Control-Block

- Blackbox konkret: exakt 7 Bit Eingang
- Ausgang: Bündel an Kontrollsignalen
- Signale fließen zu Multiplexern und Speicher-Pins
- Nächste Aufgabe: Wahrheitstabelle Zeile für Zeile lösen




**Notizen**

Sieben Opcode-Bits gehen hinein. Hinaus gehen die sieben benannten Signale mit acht Bits. Die Box decodiert nicht sämtliche RV32I-Befehle. ALU-Detail, Speicherbreite und Branch-Bedingung brauchen

zusätzliche Information. Pfeile zeigen die Richtung von Opcode zu den Signalen.
Kontrollfrage: Reichen diese Ausgänge für jal? Nein, jal fehlt im Lehrsubset. Vor dem Füllen der Tabelle klären wir, wann ein Ausgang unerheblich ist.

**Folie** 23 / 84

### Das Konzept Don't Care ($X$)

- Ingenieurswerkzeug: Don't Care ($X$)
- Beispiel: `MemtoReg` bei `sw` — Store schreibt nie ins Register File (`RegWrite = 0`)
- Welches Kabel der MUX durchlässt, ist irrelevant
- In die Wahrheitstabelle: $X$ statt 0 oder 1

```
MemtoReg bei sw:
Muss es 0 sein (ALU)?  -> Nein
Muss es 1 sein (RAM)?  -> Nein
Also: X (Don't Care)
```

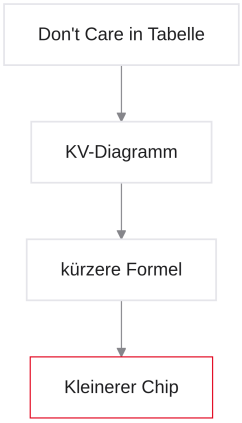
**Notizen**

Bei sw ist RegWrite gleich 0. Der Register-Schreibport übernimmt keinen Wert. Deshalb darf MemtoReg in der Tabelle 0 oder 1 sein. Beide Stellungen lassen denselben Registerzustand. X bedeutet diese Entwurfsfreiheit, nicht einen schwebenden Pegel und nicht einen abgeschalteten Draht. In der konkreten Schaltung hat der Draht trotzdem 0 oder 1. HDL, Hardware Description Language, benutzt X manchmal für einen unbekannten Simulationswert und Z für hochohmig. Das sind andere Bedeutungen.
Kontrollfrage: Ist der Draht bei X elektrisch offen? Nein. Diese Freiheit kann die Minimierung erleichtern.

**Folie** 24 / 84

## Warum $X$ Hardware spart

- $X$ gibt dem Synthese-Tool (Quartus, Vivado) Freiheit
- Im Karnaugh-Veitch-Diagramm: $X$ wahlweise als 1 oder 0 für große Blöcke
- Große Blöcke → kurze Boolesche Formeln
- Kurze Formeln → weniger Gatter, weniger Strom, höhere Taktraten




**Notizen**

X kann eine Boolesche Minimierung erleichtern. Es garantiert nicht weniger Fläche, Strom oder eine höhere Taktfrequenz. Ein Karnaugh-Veitch-Diagramm ordnet Eingaben so, dass benachbarte Einsen zusammengefasst werden. Kleines Beispiel: f(00) gleich 0, f(01) gleich 1, f(10) gleich 0, f(11) gleich X. Wählt man X als 1, gilt f gleich B. Das ist Algebra, keine Messung dieses Prozessors.
Kontrollfrage: Spart jedes X einen Transistor? Nicht nachweisbar ohne Synthese. Die leere Haupttabelle kann nun gefüllt werden.

**Folie** 25 / 84

### Die Skelett-Tabelle

- Leere Wahrheitstabelle als Arbeitsblatt
- Vier Zeilen für die wichtigsten Opcodes
- Spalten für die entscheidenden Steuersignale
- Nächste Folien: Matrix durch logisches Denken füllen

| Instruktion (Opcode) | ALUSrc | MemtoReg | RegWrite | MemRead | MemWrite | Branch | ALUOp1 | ALUOp0 |
|---|---|---|---|---|---|---|---|---|
| R-Type (`0110011`) | ? | ? | ? | ? | ? | ? | ? | ? |
| `lw` (`0000011`) | ? | ? | ? | ? | ? | ? | ? | ? |
| `sw` (`0100011`) | ? | ? | ? | ? | ? | ? | ? | ? |
| `beq` (`1100011`) | ? | ? | ? | ? | ? | ? | ? | ? |

**Notizen**

Die Spalten sind ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch und das zweibitige ALUOp. Fragezeichen bedeuten noch offen. X bedeutet begründet unerheblich. Die R-Type-Zeile meint die ausgewählten Operationen add, sub, and und or, nicht alle Encodings mit Opcode 0110011. Jede Zeile wird über Quellen, Berechnung, Ziel und Zustandsänderung hergeleitet.
Kontrollfrage: Ist ein Fragezeichen dasselbe wie X? Nein. Eine Gruppierung unterstützt die Herleitung.

**Folie** 26 / 84

### Die Spalten gruppieren

- Output-Pins mental nach CPU-Phase ordnen
- Execute (ALU): `ALUSrc`; `ALUOp1`/`ALUOp0` für die grobe ALU-Aufgabe
- Memory (RAM): `MemRead`, `MemWrite`; Branch für die Sprung-Erlaubnis
- Write-Back: `MemtoReg` und `RegWrite`

```
Signalfluss in der CPU: von links nach rechts.
Tabelle gedanklich in derselben Richtung ausfüllen.
```

**Notizen**

EX bedeutet Execute, MEM Memory Access, WB Write Back. Das sind Funktionsabschnitte dieses Single-Cycle-Datenpfads, keine eigenen Taktzyklen und keine Pipeline. Branch gehört nicht zum Datenspeicher, sondern zur PC-Auswahl. Farben allein genügen nicht; die Gruppen sind beschriftet.
Kontrollfrage: Sind EX, MEM und WB drei Takte? Hier nein. Schreib- und PC-Signale müssen präzise

festgelegt werden.

**Folie** 27 / 84

### Das eiserne Gesetz der Präzision

- Ein Fehler in der Matrix kann das System crashen lassen
- Beispiel: `RegWrite` bei `sw` vergessen und auf 1 gesetzt
- Die Schreibadresse ist das Bitfeld, das im R-Format `rd` wäre
- Bei `sw` sind das Teile des Immediate; das beschriebene Register folgt aus diesen Bits
- Der Schreibwert ist der am Write-Data-Eingang anliegende Wert, nicht unbestimmter Abfall

**Notizen**

sw t1, 12(s1) trägt im S-Immediate an den Bits 11 bis 7 den Wert 01100. Ein fälschlich aktiver Register-Schreibport würde damit x12 adressieren. Das ist kein zufälliges Register. Steht MemtoReg in diesem Fehlerbild auf 0, wäre der ALU-Adresswert der Schreibwert, nicht undefinierter Inhalt. Eine ASIC-Änderung ist nicht die zwingende Folge; ein FPGA oder ein Modell kann anders geändert werden.
Kontrollfrage: Sieht der Schreibport bei sw ein echtes rd? Nein. Anschließend prüfen wir jedes Signal einzeln.

**Folie** 28 / 84

## Die Haupt-Steuersignale

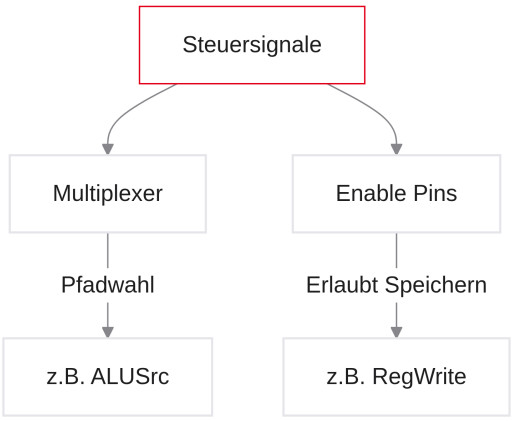
**Notizen**

Der Abschnitt wechselt von Opcode-Familien zu Bauteilwirkungen. Für jeden Pin lautet die Frage: Welche Auswahl oder Zustandsänderung steuert er, und wann ist sein Wert beobachtbar? Die sieben Namen sind die Orientierung. Ihre Begründung folgt einzeln.
Kontrollfrage: Ist ein anliegender Datenwert schon gespeichert? Nein. Die nächste Folie gruppiert die Signale.

**Folie** 29 / 84

## Die Signale im Überblick

- Jedes der sieben Steuersignale systematisch durchgehen
- Welche Signale sind Weichen (MUX)?
- Welche Signale sind Schreib-/Leserechte (Enable-Pins)?
- Warum sind Don't Cares ($X$) bei MUXen oft sinnvoll?
- Die fatale Kollision von `MemRead` und `MemWrite`



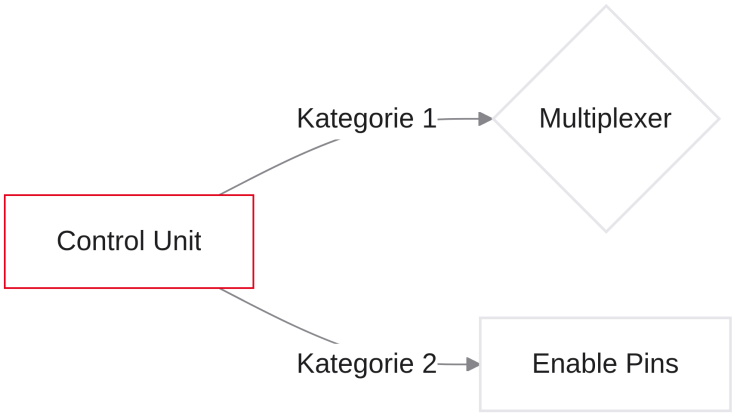

**Notizen**

Ein MUX wählt eine Datenquelle. Ein Enable gibt einen Zugriff frei. Branch ist eine Bedingungsfreigabe, nicht der Speicherzugriff. MemRead und MemWrite gleichzeitig sind in unserem Modell nicht die vorgesehene Kombination. Das ist eine Modellregel, keine universelle elektrische Kurzschlussgefahr.
Kontrollfrage: Dürfen MemRead und MemWrite im Lehrsubset beide eins sein? Nein. Als Nächstes unterscheiden wir anliegende Daten und ihre Speicherung.

**Folie** 30 / 84

## Zwei Arten von Signalen

- Im Datenpfad gibt es zwei Kategorien von Steuerleitungen
- Kategorie 1: Weichen (Multiplexer) — lenken Datenströme (z. B. `ALUSrc`)
- Eine Weiche leitet immer weiter — auch bei Müll; sie steht auf 0 oder 1
- Kategorie 2: Erlaubnis-Pins (Write-Enable) — verändern Speicherzustand (z. B. `RegWrite`)
- Enable-Pins restriktiv auf 0 halten; nur bei Bedarf 1 — sonst Datenkorruption




**Notizen**

Der MUX-Ausgang führt den ausgewählten Datenwert. Er ist kein Schreibport. Ein Register übernimmt den Wert erst bei erlaubter Freigabe und der Modellflanke. Ob ein Wert unerheblich ist, muss für jeden beobachtbaren Verbraucher geprüft werden, nicht nur für das Write-Back. Die Skizze trennt Datenpfad und Enable.
Kontrollfrage: Verändert ein MUX-Ausgang allein den Registerzustand? Nein. ALUSrc ist das erste Auswahlbeispiel.

**Folie** 31 / 84

### ALUSrc (Multiplexer 1)

- 1-Bit-Signal steuert den MUX vor dem zweiten ALU-Eingang
- 0: ALU rechnet mit Register `RD2` aus dem Register File
- 1: ALU rechnet mit der 32-Bit-Konstante aus dem ImmGen
- Reine Registeroperationen (`add`, `sub`, `and`) → `ALUSrc = 0`
- Adress-Offsets (`lw`, `sw`) → `ALUSrc = 1`

```
ALUSrc:
R-Type  -> 0   (rs2)
Load    -> 1   (Immediate-Offset)
Store   -> 1   (Immediate-Offset)
Branch  -> 0   (Register vergleichen)
```

**Notizen**

ALUSrc gleich 0 wählt RD2, den zweiten gelesenen Registerdatenwert. ALUSrc gleich 1 wählt den auf 32 Bit erweiterten Immediate. add nutzt Register plus Register. lw und sw nutzen Basis plus Offset. beq nutzt im Lehrmodell die Register für den Vergleich. addi nutzt ebenfalls die Immediate-Auswahl. In Ripes ist die Branch-Struktur anders; diese 0 gilt hier für das Lehrmodell.
Kontrollfrage: Welche ALUSrc-Stellung hat lw? 1. Danach wird die Quelle des Register-Schreibwerts gewählt.

**Folie** 32 / 84

### MemtoReg (Multiplexer 2)

- 1-Bit-Signal steuert den MUX vor dem Write-Back
- 0: ALU-Ergebnis fließt ins Zielregister
- 1: RAM-Daten fließen ins Zielregister
- RAM-Daten nur beim Load (`lw`)
- Bei R-Type muss `MemtoReg = 0` sein — sonst RAM-Schrott in der Variablen

```
MemtoReg:
R-Type  -> 0   (ALU-Ergebnis)
Load    -> 1   (RAM-Daten)
Branch  -> X   (kein Speichern)
Store   -> X   (kein Speichern)
```

**Notizen**

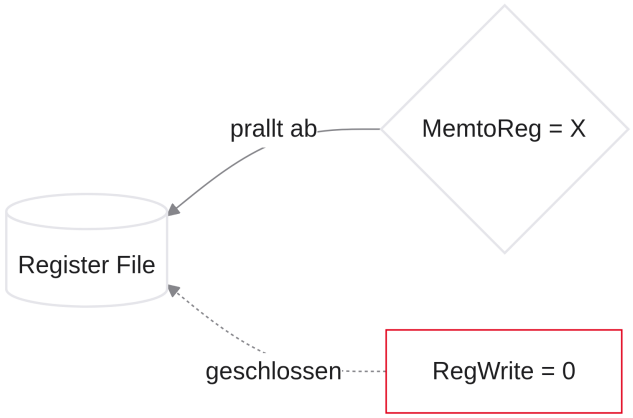
MemtoReg gleich 0 wählt das ALU-Ergebnis, gleich 1 den gelesenen Speicherwert. Lehrwerte: die ALU berechnet die Adresse 0x2004, der Speicher enthält dort 37. lw schreibt 37, nicht die Adresse. Ein falscher Datenwert ist ein bestimmter anderer Wert, kein Schrott. Ripes kann mehr als zwei Write-Back-Quellen

haben, darunter PC plus 4.
Kontrollfrage: Was schreibt lw in diesem Beispiel? 37. Ohne Registerschreiben kann die Auswahl unerheblich sein.

**Folie** 33 / 84

## MemtoReg als Don't Care ($X$)

- Bei `sw` oder `beq`: nichts wird ins Register File geschrieben
- `RegWrite = 0` — Werkbank verschlossen
- MUX-Stellung vor dem Register File ist irrelevant
- Signale prallen am geschlossenen Register File ab
- In die Wahrheitstabelle: stolzes $X$




**Notizen**

RegWrite gleich 0 deaktiviert den betrachteten Schreibport. Die Leseports können rs1 und rs2 weiter liefern. Die Registerbank ist nicht insgesamt geschlossen. Bei sw bleiben Adresse, Store-Daten und MemWrite relevant. MemtoReg gleich 0 und gleich 1 erzeugen denselben Registerzustand, weil nichts übernommen wird.
Kontrollfrage: Dürfen die Leseports bei sw weiter Daten liefern? Ja. Die nächste Folie erklärt die Schreibfreigabe genauer.

**Folie** 34 / 84

### RegWrite (Erlaubnis-Pin)

- Hochkritischer Write-Enable am Register File
- 1: Bei der nächsten positiven Taktflanke wird `rd` überschrieben
- 0: Register File schreibgeschützt
- Fehler hier sind für die Software katastrophal

```
RegWrite:
R-Type  -> 1   (Ergebnis speichern)
Load    -> 1   (geladenes Wort speichern)
Store   -> 0
Branch  -> 0
```

**Notizen**

RegWrite ist die Schreibfreigabe. Write Enable ist dasselbe Signal in manchen Bildern. rd ist die Zieladresse. Die positive Flanke ist die Annahme dieses Modells, keine ISA-Pflicht. x0 bleibt immer null, auch wenn rd gleich x0 ist. PC und Datenspeicher sind andere Zustandselemente. RegWrite gleich 0 blockiert sie nicht. Vor der Flanke kann das ALU-Ergebnis schon anliegen. Erst die Freigabe übernimmt es nach t0.
Kontrollfrage: Setzt RegWrite gleich 0 auch den PC still? Nein. Store und Branch brauchen dieses Write-Back nicht.

**Folie** 35 / 84

### Wer darf nicht schreiben?

- Zwei Familien dürfen `RegWrite` nie aktivieren
- Store (`sw`): Ziel ist der RAM; CPU-Variablen bleiben unangetastet
- Branch (`beq`): Ziel ist der PC; keine Datenberechnung/-ladung
- Control Unit zwingt `RegWrite` eisern auf 0
- Ein fälschlich aktives `RegWrite` würde die Immediate-Bits als Registeradresse sehen

**Notizen**

S- und B-Format haben kein rd. Ein naiver Adressport sieht Immediate-Bits. Die Wirkung einer falschen Freigabe ist aus Adresse und Schreibwert bestimmbar, nicht zufällig. Bei den behandelten Stores und Branches bleibt der allgemeine Registersatz unverändert. Store ändert den Datenspeicher, Branch den PC.
Kontrollfrage: Welches Register würde 01100 fälschlich wählen? x12. Nun folgen die Speicherzugriffe.

**Folie** 36 / 84

### MemRead (Erlaubnis-Pin)

- Erlaubnis-Pin für das Lesen aus dem Datenspeicher (RAM)
- 1: RAM legt Bits an der ALU-Adresse auf das Lesekabel
- 0: im Lehrmodell kein vorgesehener Lesezugriff, nicht automatisch hochohmig
- Im Single-Cycle nur bei `lw` auf 1
- Sonst Strom sparen: RAM nicht unnötig auslesen

```
MemRead:
Load          -> 1
Alle anderen -> 0
```

**Notizen**

MemRead ist die explizite Lesezugriffssteuerung des Lehrmodells. MemRead gleich 0 bedeutet hier keinen vorgesehenen Lesezugriff. Ob der Ausgang dann einen Pegel oder einen hochohmigen Zustand hat, entscheidet die konkrete Speicherimplementierung. Ripes steuert Zugriffsart und Breite oft über einen Operationsselektor, nicht zwingend über einen gleichnamigen Pin. Eine Stromersparnis ist implementationsabhängig und hier nicht gemessen.
Kontrollfrage: Bedeutet MemRead gleich 0 immer den Zustand Z? Nein. Schreiben ist ein eigener Zugriff.

**Folie** 37 / 84

### MemWrite (Erlaubnis-Pin)

- Noch kritischerer Write-Enable am RAM
- 1: Bei Taktflanke wird Write-Data in die RAM-Zelle geschrieben
- 0: Schreibschutz aktiv
- In der Basistabelle genau einmal 1: bei `sw`
- Ein falsches `MemWrite` kann den konkret adressierten Speicherwert ändern

```
MemWrite:
Store         -> 1
Alle anderen -> 0
```

**Notizen**

MemWrite braucht drei getrennte Angaben: Adresse, Schreibdaten und Freigabe. Bei sw kommt die Adresse aus der ALU und der Schreibwert aus rs2. Die Übernahme folgt dem Speichertaktmodell. Ein falsches MemWrite kann den adressierten Wert ändern. Es gibt hier nicht automatisch ein Betriebssystem, das dadurch abstürzt.
Kontrollfrage: Liegt der Store-Wert am ALU-Ergebnis? Nein, an rs2. Die nächste Folie legt die zulässigen Kombinationen fest.

**Folie** 38 / 84

### Die Gefahr: MemRead und MemWrite

- Im Lehrmodell sind `MemRead` und `MemWrite` gegenseitig ausgeschlossen
- `MemRead = 0` bedeutet hier „nicht lesen“, nicht zwingend hochohmig
- Ein elektrischer Kurzschluss gilt nur für ein konkret gezeigtes Busmodell
- Speichermodule können ein definiertes Read-during-write haben; das nutzen wir nicht

**Notizen**

Die vier Kombinationen, zuerst MemRead dann MemWrite: 00 kein vorgesehener Zugriff, 10 lesen, 01 schreiben, 11 im Kursmodell nicht verwendet. Read-during-write realer Speicher ist eine eigene Spezifikation, kein automatischer Kurzschluss. Für lw gilt 10, für sw 01, für add und beq 00.
Kontrollfrage: Welche Kombination hat sw? MemRead 0 und MemWrite 1. Die letzte Gruppe betrifft den Kontrollfluss.

**Folie** 39 / 84

### Branch (Entscheidungs-Signal)

- Weder direkter MUX noch Enable-Pin
- Hilfssignal in das AND-Gatter vor dem PC
- 1: Control Unit bestätigt — aktueller Befehl ist bedingter Sprung
- Liefert dem AND die halbe Erlaubnis für `PCSrc`
- Bei `add`, `lw`, `sw`: Branch = 0 — Sprung unmöglich

```
Branch:
beq / Branch-Familie -> 1
Alle anderen         -> 0
```

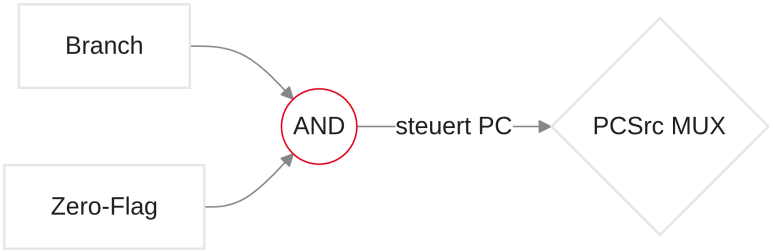
**Notizen**

Branch gleich 1 kennzeichnet einen unterstützten bedingten Sprung. Es bedeutet nicht, dass der Sprung genommen wird. PCSrc ist die Auswahl des nächsten PC. jal und jalr fehlen im Lehrsubset. Dieselbe Branch-Freigabe kann mit verschiedenen Vergleichsergebnissen verschiedenes PCSrc erzeugen.
Kontrollfrage: Ist Branch gleich 1 schon das genommene Sprungziel? Nein. Die nächste Folie setzt die Gleichung zusammen.

**Folie** 40 / 84

## Die PCSrc-Logik (Rückblick)

- Control Unit kann die finale Sprungentscheidung nicht allein fällen
- Registervergleich fehlt ihr — ALU-Zero als Assistent
- Für `beq`: PCSrc $=$ Branch $\wedge$ Zero
- Beide Signale müssen 1 sein, damit der PC umgeschaltet wird
- Andere Branches (z. B. `bne`): andere Flag-Logik (oft Branch $\wedge$ $\overline{\text{Zero}}$)




**Notizen**

Für beq gilt PCSrc gleich Branch und Zero. Zero ist eins genau bei ALU-Ergebnis null. Die vier Fälle: Branch 0 und Zero 0 oder 1 lassen den Sprung ungewählt. Branch 1 und Zero 0 ergibt PCSrc 0, also PC plus 4. Branch 1 und Zero 1 ergibt PCSrc 1. Der Offset ist relativ zur aktuellen Branch-Adresse, nicht zu PC plus 4. Lehrwerte: PC gleich 0x100, Offset plus 12. Genommenes Ziel 0x10C, nicht genommen 0x104. Das sind illustrative Adressen, keine Ripes-Startadresse. bne braucht Branch und nicht Zero. Im Ripes-Single-Cycle-Modell vergleicht ein eigener Block die Register.
Kontrollfrage: Wohin springt beq bei Gleichheit in diesem Beispiel? Nach 0x10C. Als Nächstes fassen wir die Signale zusammen.

**Folie** 41 / 84

## Zwischenstand der Tabelle

- Weichen und Enable-Pins für den Chip unter Kontrolle
- Datenverkehr und Speicherblockaden sind spezifiziert
- Noch offen: Was genau soll die ALU tun?
- Main Control hat bisher nur die groben Pfade organisiert

| Signal | Funktion | Bauteil |
|---|---|---|
| `ALUSrc` | 0=Reg, 1=Imm | MUX vor ALU |
| `MemtoReg` | 0=ALU, 1=RAM | MUX vor Write-Back |
| `RegWrite` | 1=Schreiben | Register-File-Pin |
| `MemRead/MemWrite` | 1=Aktiv | RAM-Pins |
| `Branch` | 1=Sprungbereit | AND vor PC |

**Notizen**

Die Tabelle zeigt ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite und Branch. ALUOp fehlt noch. MemRead und MemWrite sind zwei Signale. Branch ist nicht der fertige PC-Selektor. Zustand ändern RegWrite, MemWrite und die PC-Fortschaltung. ALUSrc und MemtoReg wählen nur Quellen, MemtoReg aber nur beobachtbar bei RegWrite gleich 1.
Kontrollfrage: Welches Signal schreibt den Datenspeicher? MemWrite. Nun wird die konkrete ALU-Operation bestimmt.

**Folie** 42 / 84

## Das fehlende ALU-Signal

- Warum kein simples `ALU_Addition`-Signal in der Main Control?
- Problem: der 7-Bit-Opcode
- `add` und `sub` teilen denselben Opcode `0110011`
- Main Control sieht nur die Familie — nicht Plus vs. Minus

```
; Main Control sieht nur 0110011:
add t0, t1, t2
sub t0, t1, t2
and t0, t1, t2
```

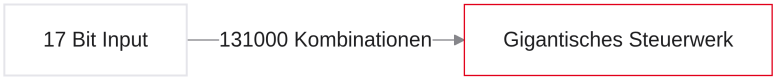
**Notizen**

add, sub und and können dieselben Registeroperanden und denselben Opcode 0110011 haben. Die Ergebnisse unterscheiden sich, weil funct-Felder verschieden sind. Ein einzelnes Addition-ja-nein-Signal reicht für das Subset nicht. Die Detaildecodierung ist eine Modellzerlegung, keine ISA-Vorschrift.
Kontrollfrage: Haben add und and denselben Opcode? Ja. Die nächste Folie quantifiziert eine naive Eingabetabelle.

**Folie** 43 / 84

## Das Problem mit R-Types

- Opcode, `funct3` und `funct7` sind zusammen viele Eingangsbits
- Eine naive Tabelle hätte $2^{17} = 131072$ Zeilen
- Die synthetisierte Schaltung listet diese Zeilen nicht physisch auf
- Don't-Care und die Zweistufigkeit verkleinern die Beschreibung, nicht per se die gemessene Fläche



**Notizen**

Sieben Opcode-Bits, drei funct3-Bits und sieben funct7-Bits sind 17 Bits. 2 hoch 17 ist 131072 Kombinationen einer naiv vollständigen Tabelle. Das ist nicht die physische Schaltung und kein Beleg, dass die Taktfrequenz einbricht. Nicht jede Kombination ist ein unterstützter Befehl. Strukturierte Decoder nutzen gemeinsame Bedingungen. Kosten hängen von der Umsetzung ab.
Kontrollfrage: Sind 131072 Zeilen die Gatterzahl? Nein. Die nächste Zerlegung trennt Familie und Funktionsbits.

**Folie** 44 / 84

## ALU Control (Zweistufige Decodierung)

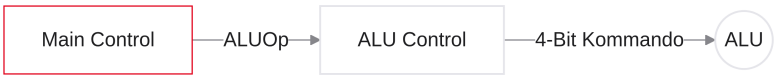
**Notizen**

ALU bedeutet Arithmetic Logic Unit. Decodierung ordnet Bitmuster einer Operation zu. Die offene Frage lautet: Wie verbindet man die grobe Familie mit den feinen Funktionsbits, ohne alles in einer unübersichtlichen Tabelle zu vermischen? Die Antwort ist eine Modellzerlegung, keine ISA-Pflicht.
Kontrollfrage: Schreibt RISC-V ALUOp vor? Nein. Die nächste Folie führt diese Brücke ein.

**Folie** 45 / 84

### Teile und Herrsche

- Ausweg aus der gigantischen Wahrheitstabelle
- Delegation von Verantwortung
- 2-Bit-Signal `ALUOp` als Brücke zwischen den Abteilungen
- Kleine ALU Control Unit als Detail-Spezialist
- Auswertung von `funct3` und `funct7`




**Notizen**

Die Main Control bildet aus dem Opcode die Kategorie ALUOp. Die ALU Control bildet daraus und aus den Funktionsbits das konkrete Kommando. Bei lw verlangt die Main Control Addition. Die ALU Control muss dann die Registerfunktionsbits nicht noch einmal als Operationsentscheid lesen. Zwei Boxen mit Bitbreiten machen die Zuständigkeit sichtbar. Eine einzige große Tabelle ist keine notwendige physische Alternative.
Kontrollfrage: Muss lw die funct7-Bits der R-Types auswerten? Für die bereits erkannte Additionsaufgabe nein. Die nächste Folie zeigt die Ein- und Ausgänge.

**Folie** 46 / 84

### Die Lösung: Zweistufige Kontrolle

- Logik in zwei separate Boxen
- Stufe 1 (Main Control): 7-Bit-Opcode; MUXe und Speicher; sendet `ALUOp`
- Stufe 2 (ALU Control): nimmt `ALUOp`, liest `funct3`/`funct7`, finales ALU-Kommando
- Zwei kleine Schaltungen sind schneller und kleiner als eine gigantische

**Notizen**

Zwei kleine Schaltungen sind nicht automatisch schneller oder kleiner. Eine zusätzliche Stufe kann Laufzeit beitragen. Der Nutzen hier ist eine prüfbare Zuständigkeit. ALUOp ist ein Zweibitbus. ALUControl ist das vierbitige Kurskommando. Der Weg ist Opcode, Main Control, ALUOp, ALU Control, plus getrennt zugeführte Funktionsfelder.
Kontrollfrage: Ist ALUOp schon der Addierbefehl? Nein. Die nächste Folie definiert die vier Kategorien.

**Folie** 47 / 84

### Das Signal ALUOp (2 Bit)

- Verlässt die Main Control Richtung ALU Control
- 2 Bit — vier grobe Zustände
- Kategorisiert die Aufgabe der ALU

| ALUOp | Bedeutung | Befehlstypen |
|---|---|---|
| `00` | ALU muss addieren (unbedingt) | `lw`, `sw`, `addi` |
| `01` | ALU muss subtrahieren (Vergleich) | `beq` |
| `10` | ALU muss rechnen (funct lesen) | R-Type |
| `11` | Reserve / ungenutzt | - |

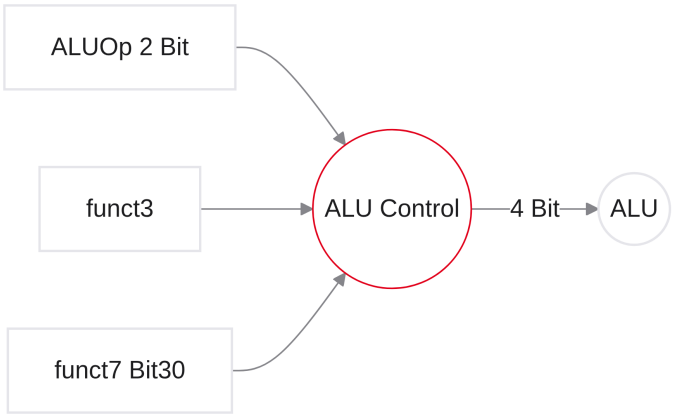
**Notizen**

ALUOp 00 verlangt ADD für lw, sw und addi. 01 verlangt im Lehrmodell SUB für den beq-Vergleich. 10 lässt die unterstützte Registeroperation aus den funct-Feldern bestimmen. 11 ist unbenutzt. Ungültige Eingaben dürfen daraus kein beliebiges Schreiben ableiten. addi mit Immediate minus 5 hat instr[30] gleich 1, weil dieses Bit zum Immediate gehört, und muss trotzdem addieren. Das Maschinenwort addi t0, x0, minus 5 ist als 0xFFB00293 zu prüfen. Diese Fassung hat den Assembler nicht ausgeführt.
Kontrollfrage: Darf Bit 30 bei addi die Subtraktion wählen? Nein. Die ALU Control übersetzt die Kategorie.

**Folie** 48 / 84

## Die kleine ALU Control Unit

- Eingänge: `ALUOp` (2), `funct3` (3), relevantes `funct7`-Bit (1)
- `funct7` nur auswerten, wenn nötig (R-Type)
- Ausgang: 4-Bit-Steuerwort in die ALU
- Definiert die Verschaltung: Plus, Minus, AND, OR




**Notizen**

Die vereinfachte Box sieht sechs Bits: zwei ALUOp, drei funct3 und instr[30]. Das ersetzt nicht die Legalitätsprüfung über Opcode und vollständiges funct7. Nur bei Bedarf auswerten darf illegale Kombinationen nicht als gültige Operationen durchlassen. Der Ausgang hat vier Bits nach der Kurskonvention 0000 AND, 0001 OR, 0010 ADD und 0110 SUB.
Kontrollfrage: Sind diese vier Bits RISC-V-Opcodes? Nein. Die einfachste Kategorie erzwingt Addition.

**Folie** 49 / 84

## Fall 1: ALUOp = 00 (Load, Store, addi)

- Chef funkt `00` — `lw` oder `sw`
- Hardware muss Base + Offset addieren
- ALU Control ignoriert funct-Bits ($X$)
- Sendet hart verdrahtet ADD (`0010`) an die ALU

```
WENN ALUOp == 00:
  Sende ADD an die ALU
  (funct-Felder irrelevant)
```

**Notizen**

ALUOp 00 erzwingt Addition für bereits gültig erkanntes lw, sw und addi. Bei lw und sw entsteht eine Adresse, bei addi ein Registerwert. Der Immediate wird vorzeichenerweitert. Lehrwerte: Basis 0x2000 plus

Offset 4 ergibt 0x2004. addi von 7 und minus 5 ergibt 2. Die funct-Felder entscheiden in dieser Kategorie nicht noch einmal die Operation.
Kontrollfrage: Addiert ALUOp 00 auch bei einem gesetzten Bit 30 von addi? Ja. Kategorie 01 nutzt im Lehrmodell eine andere Funktion.

**Folie** 50 / 84

### Fall 2: ALUOp = 01 (beq-Vergleich im Lehrmodell)

- Chef funkt `01` — bedingter Sprung
- Zwei Register subtrahieren, um Zero zu füttern
- funct-Bits wieder ignoriert
- Sendet SUB (`0110`) an die ALU

```
WENN ALUOp == 01:
  Sende SUB an die ALU
  (Zero-Flag bei Gleichheit)
```

**Notizen**

Der Untertitel gilt für den beq-Vergleich im Lehrmodell, nicht für jeden Branch und nicht für Ripes. 9 minus 9 ist 0, Zero wird 1. 9 minus 4 ist 5, Zero bleibt 0. Die ALU vergleicht. Ein separater Addierer bildet das Ziel PC plus Offset. In Ripes übernimmt ein eigener Vergleicher die Register. Der vollständige Decoder darf die Branch-Art nicht generell ignorieren.
Kontrollfrage: Berechnet die ALU hier das Sprungziel? Im Lehrmodell nein. R-Type braucht eine funktionsabhängige Auswahl.

**Folie** 51 / 84

### Fall 3: ALUOp = 10 (R-Type)

- Chef funkt `10` — Detail-Arbeit
- ALU Control erwacht und liest die funct-Felder
- Unterscheidung: `add`, `sub`, `and`, ...

```
Opcode 0110011, ALUOp = 10
-> funct3 / funct7 scannen
```

**Notizen**

funct3 und funct7 unterscheiden die unterstützten Registeroperationen. Dieselben Quellwerte können add, sub, and oder or durchlaufen. Die Pfadauswahlen bleiben gleich, das ALU-Ergebnis nicht. Erwacht ist eine Metapher und bedeutet nicht, dass alle anderen Gatter physisch abgeschaltet sind. Voraussetzung sind legale Encodings des Subsets.
Kontrollfrage: Ändert or gegenüber add den Wert von ALUSrc? Nein. Zuerst wird funct3 gelesen.

**Folie** 52 / 84

### funct3 als Wegweiser

- Bei `ALUOp = 10` ist `funct3` der primäre Wegweiser
- `funct3 = 111` — bitweises AND
- `funct3 = 110` — bitweises OR
- `funct3 = 000` — Arithmetik (Plus oder Minus?)
- Bei `000` reicht funct3 noch nicht — nächste Stufe

| ALUOp | funct3 | Aktion |
|---|---|---|
| `10` | `111` | AND |
| `10` | `110` | OR |
| `10` | `000` | Plus oder Minus? → Bit 30 |

**Notizen**

funct3 liegt in den Bits 14 bis 12. 000 ist der Addier-/Subtrahierfall, 110 ist or, 111 ist and. Bitweise AND und OR verknüpfen gleiche Positionen. Ein kurzes Beispiel 0x0F und 0xA5 ist nur ein Ausschnitt der 32-Bit-Operation. 000 allein unterscheidet add und sub nicht. Dieselben funct3-Codes bedeuten in anderen Opcode-Familien nicht dieselbe Operation.
Kontrollfrage: Reicht funct3 gleich 000 für sub? Nein. funct7 löst den Unterschied.

**Folie** 53 / 84

### funct7 als letzter Schalter

- Nur wenn `ALUOp = 10` und `funct3 = 000`
- Relevant: Bit 30 der Instruktion (in `funct7`)
- Bit 30 = 0 — `add`
- Bit 30 = 1 — `sub`
- Rätsel entschlüsselt

```
add t0, t1, t2   # Bit 30 = 0  (funct7 = 0000000)
sub t0, t1, t2   # Bit 30 = 1  (funct7 = 0100000)
```

**Notizen**

add hat funct7 gleich 0000000, sub gleich 0100000. Der sichtbare Unterschied dieser beiden Encodings bei gleichen Registern ist Bit 30. Die Aussage gilt nicht für beliebige Bitmuster. Ein MUL der M-Erweiterung kann Opcode 0110011 und funct3 000 haben, aber funct7 0000001. Ein naiver Blick nur auf Bit 30 würde es fälschlich als add lesen. Bei Immediate-Befehlen ist Bit 30 kein funct7-Bit.
Kontrollfrage: Darf jedes Muster mit Bit 30 gleich 0 als add gelten? Nein. Der Decoder erzeugt daraus das Kurs-Steuerwort.

**Folie** 54 / 84

### Das 4-Bit-ALU-Kommando

- Ergebnis der ALU Control: 4 Steuerbits in die Haupt-ALU
- Standardcodierung nach Patterson/Hennessy
- ALU-Hardware schaltet stur bei diesen Bitmustern

| 4-Bit ALU-Kommando | Hardware-Reaktion |
|---|---|
| `0000` | bitweises AND |
| `0001` | bitweises OR |
| `0010` | ADDITION |
| `0110` | SUBTRAKTION |

**Notizen**

Die vier Codes sind die Kurskonvention nach dem Lehrbuchmuster, nicht ein RISC-V-Standard. 0000 ist AND, 0001 OR, 0010 ADD, 0110 SUB. Eine mögliche ALU bildet SUB, indem sie den zweiten Operanden invertiert und den ersten Carry auf eins setzt. Das ist ein Implementierungsbeispiel. ALUOp und ALUControl bleiben zwei Tabellen.
Kontrollfrage: Ist 0110 der Opcode von sub? Nein. Die nächste Folie fasst die Zuordnung zusammen.

**Folie** 55 / 84

### Die Wahrheitstabelle der ALU Control

- Matrix der Experten-Box — als Gatter verlötet
- $X$ rettet vor unnötigen Gattern bei Load/Store/Branch

| ALUOp | Bit 30 | funct3 | Bedeutung im Lehrsubset | ALU-Output |
|---|---|---|---|---|
| `00` | $X$ | $X$ | `lw` / `sw` / `addi`, legal erkannt | `0010` (ADD) |
| `01` | $X$ | $X$ | `beq` im Lehrmodell | `0110` (SUB) |
| `10` | `0` | `000` | `add`, funct7 `0000000` | `0010` (ADD) |
| `10` | `1` | `000` | `sub`, funct7 `0100000` | `0110` (SUB) |
| `10` | $X$ | `111` | `and`, funct7 `0000000` | `0000` (AND) |
| `10` | $X$ | `110` | `or`, funct7 `0000000` | `0001` (OR) |

**Notizen**

Die Tabelle gilt für legale Eingaben des Lehrsubsets. X bei Bit 30 oder funct3 bedeutet: für diese bereits erkannte Kategorie wird das Bit nicht als weiterer Operationsentscheid gebraucht. Es ersetzt keine Prüfung des ganzen Encodings. ALUOp 00 ergibt 0010 für lw, sw und addi. 01 ergibt 0110 für beq im Lehrmodell.

10 mit funct3 000 und Bit 30 gleich 0 ist add, mit Bit 30 gleich 1 sub. funct3 111 ist and mit 0000. funct3 110 ist or mit 0001. addi t0, x0, minus 5 bleibt Addition, obwohl Bit 30 eins ist. or von 0x0F und 0xA5 ergibt im unteren Byte 0xAF. ALUOp 11 ist unbenutzt.
Kontrollfrage: Welche ALUControl-Zeile hat or? 0001. Danach bewerten wir die Zerlegung als Beschreibung.

**Folie** 56 / 84

## Warum zweistufig? (Effizienz)

- Zwei getrennte Beschreibungen: $2^7 = 128$ und $2^6 = 64$ Eingabekombinationen
- Das ist keine Messung von Fläche, Strom oder Takt
- Die 6-Bit-Tabelle ersetzt keine vollständige Legalitätsprüfung




**Notizen**

2 hoch 7 ist 128, 2 hoch 6 ist 64. Zusammen sind das 192 Eingabekombinationen zweier getrennter Beschreibungen. Das ist kein Flächen- oder Taktvergleich mit 2 hoch 17. Die kleine Tabelle enthält einen reduzierten Eingang und prüft nicht jedes illegale funct7. Der Vorteil ist Übersicht, gemeinsame Regeln und nachvollziehbare Verifikation.
Kontrollfrage: Beweisen 192 Zeilen einen höheren Takt? Nein. Das Zwischenfazit begrenzt, was fertig ist.

**Folie** 57 / 84

## Das Steuerkonzept für unser Lehrsubset steht

- Ausgewählte Pfade und sieben Hauptsignale sind spezifiziert
- Nicht die vollständige RV32I-Implementierung
- Als Nächstes: die Meistertabelle für add, lw, sw und beq

**Notizen**

Bekannt sind die ausgewählten Pfade und Steuersignale des Lehrsubsets. Eine vollständige ISA-Implementierung, ein physischer Chip und Messwerte stehen damit nicht fest. Es fehlen unter anderem weitere ALU-Operationen, Bytezugriffe, Sprünge mit Rücksprungadresse und die Behandlung nicht unterstützter Encodings. Offen bleibt die konsistente Haupttabelle.
Kontrollfrage: Ist die CPU damit vollständig gebaut? Nein. Der nächste Abschnitt führt die Signale pro Instruktion zusammen.

Folie 58 / 84

## Signal-Trace (Die Meistertabelle)

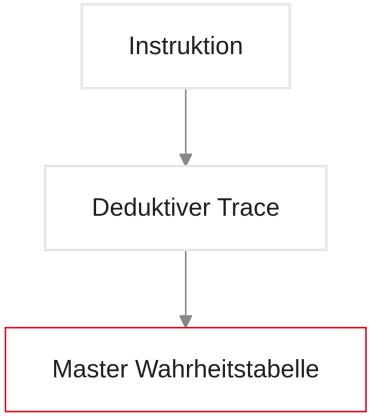
**Notizen**

Ein Trace verfolgt Daten und Signale Schritt für Schritt. Dieselbe Befehlswirkung wird in die sieben Hauptsignale übersetzt. Die Matrix gehört zum Lehrmodell. Ripes wird später funktional abgeglichen, nicht bitweise gleichgesetzt.
Kontrollfrage: Ist der Trace eine Ripes-Portliste? Nein. Die nächste Folie beschreibt die Herleitungsmethode.

Folie 59 / 84

## Die Meistertabelle

- Alle Hebel in einer Matrix
- Deduktiv herleiten statt auswendig lernen
- Trace durch R-Type, Load, Store und Branch
- Don’t Cares optimieren die Tabelle
- Referenzblatt für die Ripes-Laborsimulation




**Notizen**

Die Reihenfolge lautet: Quellen, ALU-Aufgabe, Speicherzugriff, Write-Back, PC-Änderung. Ein X braucht eine Begründung. Die Tabelle ist nicht die unveränderte Ripes-Portbelegung. Eine leere Zeile fragt: Woher kommen die Operanden, was rechnet die ALU, wer schreibt, und ändert sich der PC abweichend von plus 4?
Kontrollfrage: Darf ein X ohne Begründung stehen? Nein. Das Arbeitsblatt legt die Spalten fest.

**Folie** 60 / 84

### Die finale Matrix (Das Skelett)

- Control-Tabelle nach Patterson/Hennessy
- Ausfüllen durch logisches Denken, nicht auswendig
- Vier Befehlsfamilien (Zeilen), acht Ausgangssignale (Spalten)
- Datenpfad aus Einheit 5 im geistigen Auge behalten

| Instruk-tion | ALUSrc | Mem-toReg | Reg-Write | Mem-Read | MemWrite | Branch | ALU-Op1 | ALU-Op0 |
|---|---|---|---|---|---|---|---|---|
| R-Type | ? | ? | ? | ? | ? | ? | ? | ? |
| `lw` | ? | ? | ? | ? | ? | ? | ? | ? |
| `sw` | ? | ? | ? | ? | ? | ? | ? | ? |
| `beq` | ? | ? | ? | ? | ? | ? | ? | ? |

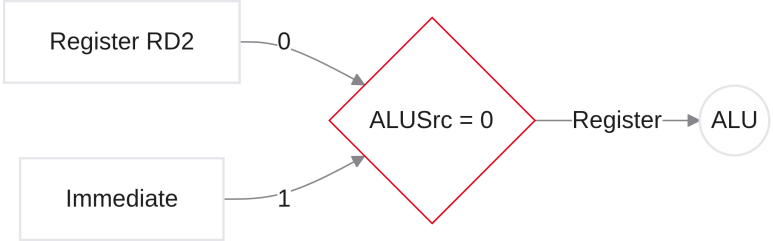
**Notizen**

Sieben Hauptsignale ergeben acht Bits, weil ALUOp zwei Bit hat. ALUOp1 ist das höhere, ALUOp0 das niedrige Bit. Vier Zeilengruppen sind ausgewählte Befehle, keine vollständigen Familien. R-Type meint add, sub, and und or. Die Herleitung fragt für jede Spalte nach der beobachtbaren Wirkung.
Kontrollfrage: Sind es acht unabhängige Main-Control-Ausgänge? Nein, sieben Namen und acht Bits. Der erste Trace beginnt mit add.

**Folie** 61 / 84

### Trace: R-Type (`add`) — ALUSrc

- Zeile für R-Type (Opcode `0110011`)
- Womit rechnet die ALU? Mit dem zweiten Register (`rs2`)
- MUX vor der ALU schaltet das Register-Kabel durch
- `ALUSrc = 0`
- ImmGen bei R-Type abgeklemmt



**Notizen**

add t0, t1, t2 mit t1 gleich 7 und t2 gleich 3. rs1 und rs2 sind die Adressfelder, 7 und 3 die gelesenen Werte. ALUSrc gleich 0 wählt RD2 gleich 3. Der ImmGen muss nicht physisch abgeschaltet sein. Sein Wert wird für diesen ALU-Eingang nicht gewählt. Das Ergebnis der Addition wird 10.
Kontrollfrage: Sieht die ALU die Registernummern oder die Werte? Die Werte. Das Ergebnis muss ins Zielregister.

**Folie** 62 / 84

### Trace: R-Type — Speichern

- Ergebnis verlässt die ALU und muss gespeichert werden
- Schreiben wir ein Register (`rd`)? Ja → `RegWrite = 1`
- Welches Signal speichern? Das ALU-Ergebnis
- MemtoReg klemmt den RAM ab → `MemtoReg = 0`

```
R-Type:
RegWrite = 1
MemtoReg = 0
```

**Notizen**

Speichern meint hier das Register-Write-Back, nicht den Datenspeicher. MemtoReg gleich 0 führt die 10 zum Datenport von t0. RegWrite gleich 1 erlaubt die Übernahme an der Modellflanke. Der vorherige Inhalt von t0 ist beliebig, solange er vor dem Befehl nicht gelesen wird. Der Datenspeicher wird nicht beschrieben.
Kontrollfrage: Welcher Wert steht nach der Flanke in t0? 10. Die übrigen Signale schließen die Zeile ab.

**Folie** 63 / 84

### Trace: R-Type — RAM und ALUOp

- Benutzen wir den RAM? Nein → `MemRead = 0`, `MemWrite = 0`
- Bedingter Sprung? Nein → `Branch = 0`
- ALU-Kommando: Entscheidung delegieren → `ALUOp = 10`
- Erste Zeile der Meistertabelle ist komplett

| Instruk-tion | ALUSrc | Mem-toReg | Reg-Write | Mem-Read | MemWrite | Branch | ALU-Op1 | ALU-Op0 |
|---|---|---|---|---|---|---|---|---|
| R-Type | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 |

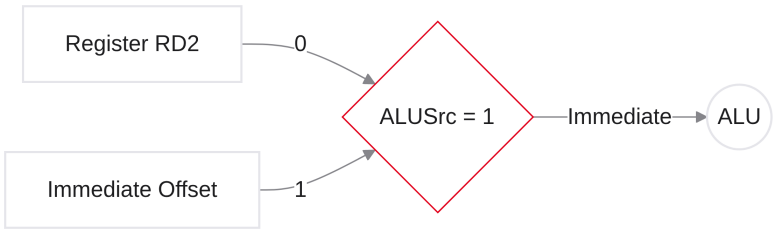
**Notizen**

Die Zeile ist ALUSrc 0, MemtoReg 0, RegWrite 1, MemRead 0, MemWrite 0, Branch 0, ALUOp 10. RAM bedeutet Random Access Memory. Lesen und Schreiben des Datenspeichers sind aus. Branch gleich 0 wählt den sequenziellen PC-Pfad, im Modell ohne C plus 4. ALUOp 10 und die Felder von add ergeben ALUControl 0010. Der Registerzustand ändert sich, der Speicher nicht.
Kontrollfrage: Welches ALUOp hat add? 10. Load verwendet Immediate und Speicher-Write-Back.

**Folie** 64 / 84

### Trace: I-Type (`lw`) — ALUSrc

- Load Word (Opcode `0000011`)
- Adresse = Base + Offset → Immediate nötig
- ALUSrc schaltet ImmGen durch
- `ALUSrc = 1`




**Notizen**

Lehrwerte, nicht ein vorhandener Simulatorzustand: s1 gleich 0x2000, lw t0, 4(s1), Speicherwort an 0x2004 gleich 37. ALUSrc gleich 1 wählt den vorzeichenerweiterten Offset plus 4. Die ALU bildet die Adresse 0x2004. Das ist noch nicht das geladene Wort.
Kontrollfrage: Was liegt nach der Addition an der Speicheradresse an? 0x2004. Der Speicher liefert nun den Inhalt.

**Folie** 65 / 84

### Trace: I-Type (`lw`) — RAM

- Daten auslesen? Ja → `MemRead = 1`
- `MemWrite = 0` (Schutz)
- Letzte Weiche: RAM-Daten zum Register → `MemtoReg = 1`

```
lw:
MemRead  = 1
MemWrite = 0
MemtoReg = 1
```

**Notizen**

MemRead gleich 1 liefert in diesem Modell den Wert 37. MemWrite gleich 0 lässt den Speicher unverändert. MemtoReg gleich 1 wählt 37 statt der Adresse 0x2004. Ein Word ist hier 32 Bit. Die Lesezugriffsfreigabe ist nicht dasselbe wie ein bestimmter elektrischer Ausgangszustand. Adresse und Daten sind verschieden beschriftet.
Kontrollfrage: Was darf nicht nach t0 geschrieben werden? Die Adresse 0x2004. Die Schreibfreigabe vollendet den Load.

**Folie** 66 / 84

### Trace: I-Type (`lw`) — Reg und ALUOp

- In `rd` schreiben? Ja → `RegWrite = 1`
- Sprung? Nein → `Branch = 0`
- ALU muss Adresse addieren → `ALUOp = 00`
- Zweite Zeile steht

| Instruk-tion | ALUSrc | Mem-toReg | Reg-Write | Mem-Read | MemWrite | Branch | ALU-Op1 | ALU-Op0 |
|---|---|---|---|---|---|---|---|---|
| `lw` | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |

**Notizen**

Die Zeile ist 1, 1, 1, 1, 0, 0, 00. RegWrite gleich 1 schreibt t0. ALUOp 00 hat die Adresse addiert. Nach der Übernahme ist t0 gleich 37. s1 und der Speicher bleiben gleich. Der PC läuft im Modell um vier Byte weiter. Wäre rd gleich x0, bliebe x0 null.
Kontrollfrage: Welches MemtoReg hat lw? 1. Store teilt die Adressbildung, nicht das Register-Write-Back.

**Folie** 67 / 84

### Trace: S-Type (`sw`)

- Store Word (Opcode `0100011`)
- Adresse wieder Base + Offset → `ALUSrc = 1`
- In den RAM schreiben? Ja → `MemWrite = 1`
- `MemRead = 0` im Modell: dieser Zugriff liest nicht

```
sw:
ALUSrc   = 1
MemWrite = 1
MemRead  = 0
```

**Notizen**

sw t1, 4(s1) mit s1 gleich 0x2000, t1 gleich 10 und bisherigem Speicherinhalt 37. ALUSrc gleich 1 und ALUOp 00 erzeugen die Adresse 0x2004. Die Store-Daten 10 kommen getrennt aus rs2. MemWrite gleich 1 und MemRead gleich 0 sind die Zugriffsregeln. Beide Pfade bleiben in derselben Abbildung sichtbar.
Kontrollfrage: Welcher Wert wird gespeichert? 10, nicht 37. Das nicht benötigte Write-Back erklärt das X.

**Folie** 68 / 84

### Trace: S-Type (`sw`) — Don't Cares

- Werkbank-überschreiben? Nein → `RegWrite = 0`
- MemtoReg bei geschlossener Tür wirkungslos → $X$
- ALU wieder Adressaddition → `ALUOp = 00`

| Instruktion | ALUSrc | MemtoReg | RegWrite | MemRead | MemWrite | Branch | ALUOp1 | ALUOp0 |
|---|---|---|---|---|---|---|---|---|
| `sw` | 1 | $X$ | 0 | 0 | 1 | 0 | 0 | 0 |

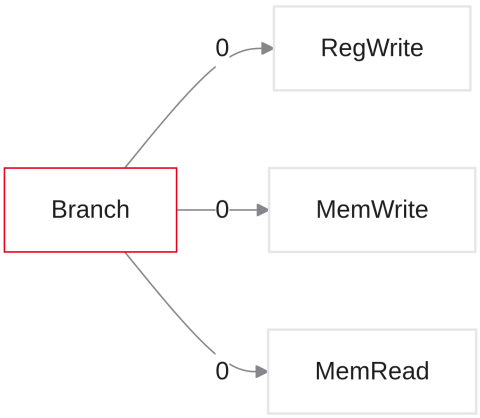
**Notizen**

Die Zeile ist 1, X, 0, 0, 1, 0, 00. Nach der Übernahme enthält 0x2004 den Wert 10. Die Register bleiben unverändert. MemtoReg darf 0 oder 1 sein, weil RegWrite 0 ist. MemWrite bleibt zwingend 1. S-Type hat kein rd. Das X gilt für dieses Modell und ist keine Freiheit für die Speichersteuersignale.
Kontrollfrage: Darf MemWrite hier X sein? Nein. Branch hat ebenfalls kein Register-Write-Back, ändert aber den PC anders.

**Folie** 69 / 84

### Trace: B-Type (beq)

- Branch (Opcode `1100011`)
- Vergleich zweier Register → `ALUSrc = 0`
- Kein Speichern in RAM oder Register File
- `MemWrite = 0, MemRead = 0, RegWrite = 0`




**Notizen**

RegWrite, MemRead und MemWrite sind 0. Der PC wird trotzdem aktualisiert. Zwei Fälle: gleiche Register ergeben Zero 1, ungleiche Zero 0. ALUSrc gleich 0 gilt für den SUB-Vergleich im Lehrmodell, nicht automatisch in Ripes. PC 0x100 und Offset plus 12 bleiben das Zahlenbeispiel. Enable null heißt nicht, dass kein Zustandselement weiterschaltet.
Kontrollfrage: Bleibt der PC bei beq stehen? Nein. Bedingung und Freigabe bestimmen den nächsten PC.

**Folie** 70 / 84

### Trace: B-Type (beq) — Control und $X$

- Branch-Weiche vor dem PC freigeben → `Branch = 1`
- MemtoReg wieder wirkungslos (`RegWrite = 0`) → $X$
- ALU: zwingende Subtraktion → `ALUOp = 01`
- Meistertabelle vollständig deduziert

| Instruk-tion | ALUSrc | Mem-toReg | Reg-Write | Mem-Read | MemWrite | Branch | ALU-Op1 | ALU-Op0 |
|---|---|---|---|---|---|---|---|---|
| `beq` | 0 | $X$ | 0 | 0 | 0 | 1 | 0 | 1 |

**Notizen**

Die Zeile des Lehrmodells ist 0, X, 0, 0, 0, 1, 01. Bei Gleichheit ist PCSrc 1 und das Ziel 0x10C. Bei

Ungleichheit ist PCSrc 0 und der nächste PC 0x104. Branch bleibt in beiden Fällen 1. MemtoReg gleich X hängt an RegWrite gleich 0, nicht an der Branch-Bedingung. Der SUB-Vergleich und die Zieladdition sind getrennt.
Kontrollfrage: Ist Branch bei einem nicht genommenen beq null? Nein. Die vier Zeilen werden zusammengeführt.

**Folie** 71 / 84

### Die ausgefüllte Meistertabelle

- Referenz des Lehrmodells, nicht eine 1:1-Porttabelle von Ripes
- $X$ = für diese Wirkung unerheblich, kein Anzeigewert
- Die R-Type-Zeile gilt für die unterstützten legalen Operationen add, sub, and, or

| Instruktion | ALUSrc | MemtoReg | RegWrite | MemRead | MemWrite | Branch | ALUOp1 | ALUOp0 |
|---|---|---|---|---|---|---|---|---|
| R-Type | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| `lw` | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| `sw` | 1 | $X$ | 0 | 0 | 1 | 0 | 0 | 0 |
| `beq` | 0 | $X$ | 0 | 0 | 0 | 1 | 0 | 1 |
| `addi` | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |

**Notizen**

Die Basiszeilen bleiben: R-Type 0,0,1,0,0,0,10; lw 1,1,1,1,0,0,00; sw 1,X,0,0,1,0,00; beq 0,X,0,0,0,1,01. addi ist die zusätzliche Zeile 1,0,1,0,0,0,00. X ist tabellarische Unerheblichkeit. Die R-Type-Zeile gilt für die legalen unterstützten Operationen. Keine Zelle ist ein Ripes-Bit gleichen Namens. Pro Zeile gilt: Quelle, Operation, Speicher, Write-Back, PC.
Kontrollfrage: Welche Zeile schreibt aus dem Speicher? lw. Der Praxisteil prüft Wirkung und Simulatorstruktur.

**Folie** 72 / 84

## Praxis: Signal-Analyse in Ripes

**Notizen**

Ripes ist ein Lehrsimulator für RISC-V, nicht die Hardware dieses Lehrmodells. Die Frage lautet, wie dieselbe ISA-Wirkung bei anderen internen Pfaden erkannt wird. Nötig sind die Modellwahl, das Layout und der Beobachtungszeitpunkt vor der Taktflanke.
Kontrollfrage: Beweist ein Ripes-Kabel unsere Tabellenbits? Es zeigt das Ripes-Modell. Der Vergleich wird ausdrücklich.

**Folie** 73 / 84

### Vom Papier in den Simulator

- Dieselbe ISA-Wirkung, andere interne Pfade
- Lehrmodell: ALU subtrahiert zum Vergleich, ein zweiter Addierer bildet das Sprungziel
- Ripes-Single-Cycle laut `rvss.h`: separater Registervergleicher; das ALU-Ergebnis kann das Ziel liefern
- Keine Behauptung, die Steuerbits seien identisch

**Notizen**

Im Lehrmodell subtrahiert die ALU zum Vergleich, und ein zweiter Addierer bildet PC plus Offset. In der offiziellen Ripes-Datei rvss.h hat der Single-Cycle-Prozessor einen separaten Branch-Vergleicher mit den beiden Registerwerten. Das ALU-Ergebnis kann das Sprungziel liefern. Die ALU-Operandenauswahl kann Register, PC und Immediate unterscheiden. Das Write-Back kann zusätzlich PC plus 4 wählen. Befehls- und Datenzugriff sind an denselben zugrunde liegenden Speicher angebunden. Dieselbe ISA-Wirkung bedeutet nicht dieselben Selektorbits. Eine 1:1-Zuordnung gleichnamiger Signale wird nicht behauptet. Diese Fassung hat die Oberfläche nicht ausgeführt.
Kontrollfrage: Bedeutet Branch in Ripes automatisch ALUOp 01? Nein. Das Laborziel ist eine überprüfbare Vorhersage.

**Folie** 74 / 84

### Das Ziel im Labor

- Debugger: unerklärliche Registerzustände
- Ursache: ABI, Hardware, Tippfehler?
- Signalleitungen während des Clock-Ticks lesen
- Methode: Befehl ansehen → erwartete 1/0 → Linie prüfen

```
Ein Ingenieur klickt nicht auf Run und hofft.
Ein Ingenieur formuliert eine These und verifiziert sie.
```

**Notizen**

ABI bedeutet Application Binary Interface. Hier genügen die Alias t0 gleich x5, t1 gleich x6 und t2 gleich x7. Ein unerwarteter Wert kommt oft von fehlender Initialisierung, falscher Instruktion, falschem Modell oder dem falschen Zeitpunkt. Vorgehen: aktuellen PC und decodierte Instruktion lesen, Wirkung vorhersagen, konkrete Ports lesen, Zustand nach genau einem Takt prüfen.
Kontrollfrage: Ist jedes dunkle Kabel ein Hardwarefehler? Nein. Zuerst wird die passende Ansicht eingeordnet.

**Folie** 75 / 84

## Wo ist die Control Unit in Ripes?

- RV32 Single Cycle wählen, C-Erweiterung aus, Extended-Layout einschalten
- Portnamen und Werte ablesen, nicht eine feste Bildschirmposition
- Diese Fassung hat die Oberfläche nicht live geprüft; die Struktur folgt der offiziellen Quelle `rvss.h`

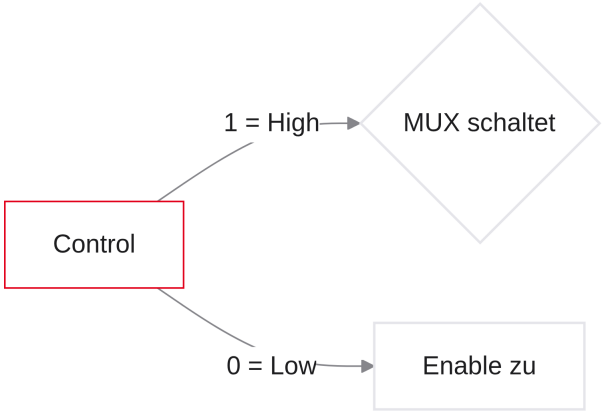
**Notizen**

Wählen Sie den RV32-Single-Cycle-Prozessor, schalten Sie die C-Erweiterung aus und das Extended-Layout ein. Die genauen Menünamen können sich ändern. Ports werden über Namen und Werte identifiziert, nicht über eine geschätzte Mitte des Bildschirms. Manche Steuerungen sind mehrbitige Selektoren. Ein UI-Test dieser Fassung liegt nicht vor. Die Strukturaussage stützt sich auf die genannte Quelldatei.
Kontrollfrage: Liegt die Control immer mittig oben? Das ist nicht belegt. Als Nächstes zählt der angezeigte Wert, nicht nur eine Farbe.

**Folie** 76 / 84

## Grüne vs. graue Leitungen

- Maßgeblich sind Portname, Breite und der angezeigte Wert
- Eine 0 kann die richtige MUX-Quelle auswählen
- $X$ aus der Tabelle ist kein dritter Anzeigewert; 0 und 1 sind bei begründetem Don’t Care beide möglich




**Notizen**

Farben können eine 1 oder 0 unterstützen, ersetzen aber nicht Portname, Breite und Zahl. Eine 0 an einem MUX kann die richtige Quelle wählen und ist dann nicht inaktiv im fachlichen Sinn. X aus der Papiertabelle erscheint nicht als dritte Farbe. Zeigt der Simulator 0 oder 1, sind beide akzeptabel, wenn der Don’t Care begründet ist und keine verbotene Wirkung entsteht.
Kontrollfrage: Muss ein graues Kabel ein Fehler sein? Nein. Eine initialisierte Vorhersage übt die Methode.

**Folie** 77 / 84

### Labor-Methode: Paper & Pencil

- Drei mentale Vorhersagen vor dem Klick
- Übung 1 (`sub`): Wie leuchten `RegWrite` und `MemWrite`?
- `RegWrite` grün (1) — ALU-Ergebnis speichern
- `MemWrite` grau (0) — RAM unberührt

```
t1 = 9, t2 = 4
sub t0, t1, t2
# Erwartung im Lehrmodell: t0 = 5
# RegWrite = 1, MemWrite = 0, ALUControl = SUB
```

**Notizen**

t1 gleich 9 und t2 gleich 4 sind gesetzte Lehrwerte. sub t0, t1, t2 schreibt 5 nach t0. t0 ist x5, t1 ist x6, t2 ist x7. Vor dem Clock werden Instruktion und stabile Steuerwerte notiert. Erwartet werden RegWrite aktiv, Speicher-Schreibfreigabe inaktiv und die Subtraktion. Die konkreten Ripes-Portcodes werden am Modell abgelesen und nicht aus der Lehrbuchtabelle erfunden. Nach dem Clock ist t0 gleich 5, der Speicher unverändert.
Kontrollfrage: Welcher Wert steht in t0? 5. Ein Load prüft eine andere Write-Back-Quelle.

**Folie** 78 / 84

### Vorhersage-Übung: `lw`

- Übung 2 (`lw`): Stellung von `ALUSrc` und `MemtoReg`?
- Beide MUX-Steuerungen auf 1 (grün)
- `ALUSrc = 1`: Offset in die ALU
- `MemtoReg = 1`: RAM-Daten ins Register

```
# Lehrwerte, nicht ein automatisch vorhandener Simulatorzustand
# zwei Wörter ab Label daten, s1 zeigt darauf
lw t0, 4(s1)
# Adresse = Basis + 4, MemtoReg wählt den Speicherwert, nicht die Adresse
```

**Notizen**

Zwei definierte Wörter stehen ab einem Label. s1 wird mit la darauf gesetzt. la kann mehrere echte Befehle erzeugen und ist nicht der Beobachtungsschritt. lw t0, 4(s1) lädt das zweite Wort, dessen Wert von der Adresse verschieden ist. Der Offset 4 ist ein Byteoffset eines ausgerichteten Wortes. Die Vorhersage: Basis plus 4, Load-Zugriff, Speicher-Write-Back, t0 erhält den zweiten Wortinhalt. Den Ripes-Selektor zeigt das Modell.
Kontrollfrage: Ist 4 der Elementindex? Nein, der Byteoffset. Branch verlangt zusätzlich den Kontrollfluss.

**Folie** 79 / 84

## Vorhersage-Übung: `bne`

- Im Lehrmodell: Branch = 1 und PCSrc = Branch UND NOT Zero
- In Ripes nicht als „ALU subtrahiert“ ablesen
- Der Vergleich sitzt in einem eigenen Block; das Ziel kann über die ALU laufen
- Ein nicht genommener Sprung ist trotzdem eine ausgeführte Branch-Instruktion

```
# Lehrmodell-Erweiterung, kein Ripes-Portwert:
bne t0, t1, Label
# Branch = 1
# genommen nur bei Ungleichheit
```

**Notizen**

bne bedeutet Branch if Not Equal. Im erweiterten Lehrmodell gilt PCSrc gleich Branch und nicht Zero. In Ripes ist das nicht die Beobachtung ALU subtrahiert. Dort vergleicht ein eigener Block, und die Zielberechnung kann über die ALU laufen. Zwei Lehrfälle: gleiche Werte springen nicht, ungleiche schon. Branch bleibt 1, auch wenn der Sprung nicht genommen wird. Register und Label müssen initialisiert sein. Kontrollfrage: Ist ein nicht genommener bne untätig? Nein, die Bedingung wurde geprüft. Danach folgt eine gedachte falsche Steuerung.

**Folie** 80 / 84

## Fehleranalyse: Typo in der Matrix

- Gedachte Decoderänderung, kein Fehler, den unverändertes Ripes bei `add` ausführt
- Beispiel: `t1 = 0x2000`, `t2 = 4`, falsches `MemWrite = 1`
- Adresse `0x2004` würde mit dem Store-Datenwert 4 beschrieben
- `t0` kann trotzdem `0x2004` erhalten

**Notizen**

Das Beispiel ist eine gedachte Änderung der Lehrsteuerung, kein Fehler, den normales add in unverändertem Ripes erzeugt. t1 gleich 0x2000 und t2 gleich 4 ergeben die ALU-Adresse 0x2004. Bei falschem MemWrite gleich 1 würde der Speicher an dieser ausgerichteten Adresse den Wert 4 aus rs2 erhalten. t0 kann gleichzeitig 0x2004 bekommen. Die Wirkung ist bestimmt, nicht zufällig. Ein Betriebssystemabsturz oder ein Kurzschluss folgt daraus nicht.
Kontrollfrage: Welcher Speicherwert würde geschrieben? 4. Eine gültige Erweiterung ist addi.

**Folie** 81 / 84

## Labor-Aufgabe 1: Transferleistung

- Bisher nur reine Basis-Befehle
- Aufgabe: Signale für `addi` (Add Immediate) herleiten
- Tipp: liest Konstante (wie `lw`), speichert ALU-Ergebnis (wie `add`)
- Auf Papier ausfüllen, dann in Ripes an den Kabeln prüfen

| Instruk-tion | ALUSrc | Mem-toReg | Reg-Write | Mem-Read | MemWrite | Branch | ALU-Op1 | ALU-Op0 |
|---|---|---|---|---|---|---|---|---|
| `addi` | ? | ? | ? | ? | ? | ? | ? | ? |

**Notizen**

Die sichtbare Tabelle bleibt für die Studierenden offen. Die Lösung ist ALUSrc 1, MemtoReg 0, RegWrite 1, MemRead 0, MemWrite 0, Branch 0, ALUOp 00. Der Opcode von addi ist 0010011, funct3 gleich 000. Plus 5 und minus 5 sind beide Additionen. minus 5 wird zu 0xFFFFFFFB. instr[30] ist dabei 1 und wählt keine Subtraktion. andi und ori addieren nicht. Sie brauchen eine andere Detaildecodierung und stehen nicht in dieser einen Zeile.
Kontrollfrage: Welches MemtoReg hat addi? 0. Der Abgleich braucht den richtigen Zeitpunkt.

**Folie** 82 / 84

## Verifikation in Ripes

- Reset, zum markierten Maschinenbefehl gehen
- Vor dem Clock: PC und Instruktion notieren, Steuerwerte lesen
- Genau einen Clock ausführen, danach Register, Speicher und PC prüfen
- Nach dem Clock können die Leitungen schon zur nächsten Instruktion gehören
- Single Cycle hat keine eigene Control-Phase

```
Gruenes Kabel am ALUSrc-MUX  -> 1 in der Tabelle
Graues Kabel                 -> 0 in der Tabelle
```

**Notizen**

Single Cycle hat keine Pipeline-Control-Phase. Zuerst zurücksetzen und initialisieren. Vor dem Clock den PC und die Maschineninstruktion notieren und die stabilen Portwerte lesen. Dann genau einen Clock ausführen und Register, Speicher sowie den nächsten PC prüfen. Danach können die Leitungen schon zur folgenden Instruktion gehören. Bei Pseudoinstruktionen zählt das Disassembly, nicht jede Quelltextzeile als ein Befehl.
Kontrollfrage: Dürfen Sie die Steuerwerte nach dem Clock noch dem abgeschlossenen Befehl zuordnen? Nicht ohne Prüfung. Das Programm übt mehrere Write-Back-Quellen.

**Folie** 83 / 84

## Labor-Aufgabe 2: Der MUX-Beobachter

- Vollständiges Programm in `beispiele/vl6_mux_beobachter.s`
- Initialisierung, Store, Load und zwei `beq`-Ausgänge
- Am Prüfpunkt: `t0 = 5, t1 = 10, t2 = 10, t3 = 1`, Speicherwort = 10
- `$X$` nur in der Papiertabelle; der Simulator zeigt einen konkreten Wert
- Abschluss in Ripes: Dienst 10 in `a7`, dann `ecall`

```
Fokus auf ein einziges Bauteil hilft gegen
Reizueberflutung im Diagramm.
```

**Notizen**

Das Programm in beispiele/vl6_mux_beobachter.s initialisiert slot mit einem Wort 0. la s1, slot setzt den Zeiger. la kann mehrere Instruktionen sein. Danach: addi t0, x0, 5 schreibt 5. add t1, t0, t0 schreibt 10. sw t1, 0(s1) schreibt 10 in den Speicher, MemtoReg ist X. lw t2, 0(s1) lädt 10, MemtoReg ist 1. beq t0, t1 springt nicht, weil 5 ungleich 10 ist. addi t3, x0, 1 schreibt 1. beq t1, t2 springt, weil beide 10 sind, und überspringt addi t3, x0, 99. Am Prüfpunkt gelten t0 gleich 5, t1 gleich 10, t2 gleich 10, t3 gleich 1 und das Speicherwort 10. Die konkreten PC-Adressen kommen aus dem Disassembly, nicht aus einer erfundenen Startadresse. Der Ripes-Abschluss setzt a7 auf 10 und führt ecall aus. Das ist die Ripes-Konvention, nicht der Venus-Dienst in a0. la und ecall liegen außerhalb der sieben-Signal-Matrix.
Kontrollfrage: Welchen Wert hat t3, wenn der zweite beq genommen wird? 1. Die Schlussfolie bündelt die Prüfung.

**Folie** 84 / 84

## Q&A und Hands-On

- Ripes starten
- Meistertabelle als Kompass nutzen
- Unerwartetes Verhalten: Logik der Hardware ergründen
- Viel Erfolg bei der Entschlüsselung des Steuerwerks




**Notizen**

Q&A bedeutet Questions and Answers, Hands-On die praktische Übung. Der Weg ist Befehlsfeld, Decoder, Auswahl oder Freigabe, berechneter Wert, Zustandsänderung. Fünf Kurzantworten: Es sind sieben Signale und acht Bits. MemtoReg ist bei sw ein X, weil RegWrite 0 ist. Branch ist die Freigabe, PCSrc die fertige Auswahl. addi mit minus 5 addiert trotz Bit 30 gleich 1. Der Ripes-Branch vergleicht in einem eigenen Block, das Lehrmodell nutzt ALU-SUB. Einheit 7 behandelt Pipelining. Dort werden Execute, Memory und Write Back auf mehrere Takte verteilt. Zusätzliche Steuerfragen entstehen. Diese Folie behauptet nicht,

jene Einheit schon ausgeführt zu haben.
Kontrollfrage: Ist PCSrc ein achter Main-Control-Ausgang? Nein. Damit ist das Lehrsubset steuerbar und vom Simulator unterscheidbar.